Resumen 2do Parcial ISW - 2026 2C

- Recomiendo pegarle una leída al primer resumen, tienen mucha relación.
- Los únicos temas que dió en clases fueron: Lean y Kanban - Métricas - Testing en ambientes ágiles. Diría que prioricen eso pero con Meles nunca se sabe.

Unidad 2: Gestión Lean Agile de Productos de Software..... 2
Filosofía Lean en el contexto del desarrollo de software (PPT 10)..... 2
7 principios Lean..... 2
Los 7+1 desperdicios de Lean..... 4
Método Kanban en el contexto del desarrollo de software (PPT 10)..... 5
6 prácticas de Kanban..... 6
Procedimiento para aplicar Kanban..... 7
9 Valores de Kanban..... 8
Principios directores..... 9
6 Principios fundacionales..... 10
Métricas en los 3 enfoques (PPT 11)..... 11
3 dominios de métricas..... 11
Métricas en los 3 enfoques..... 13
Unidad 3: Gestión de software como producto..... 15
Prácticas continuas..... 15
Unidad 4: Aseguramiento de Calidad de Proceso y de Producto..... 17
Conceptos generales sobre calidad (PPT 12)..... 17
Calidad en el software..... 20
Calidad en el producto..... 20
Modelos de calidad de producto..... 20
Calidad del proceso..... 21
Procesos..... 22
Actividades relacionadas con el Aseguramiento de la Calidad del Software..... 23
Lineamientos para la implementación de modelos de calidad en las organizaciones..... 24
2 Modelos para la mejora de procesos (IDEAL y SPICE):..... 24
Modelo de calidad para evaluar procesos (CMMI)..... 25
Relación CMMI con ágil..... 27
Auditorías al Software: Auditorías de Proyecto y Auditorías al Grupo de Calidad. (PPT 14)..... 29
3 Roles en una auditoría..... 30
Proceso y etapas de Auditorías..... 30
Calidad de Producto: Planificación de pruebas para el software- Niveles y tipos de pruebas para el software. Técnicas y herramientas para probar software. (PPT 13)..... 32
Testing en ambientes Ágiles (PPT 09)..... 36
Principios del testing..... 37
Conceptos importantes..... 40
Niveles de prueba..... 41
4 Ambientes / entornos de testing y su proceso..... 43
Artefactos de testing..... 45
Tipos de prueba..... 48

Unidad 2: Gestión Lean Agile de Productos de Software

Filosofía Lean en el contexto del desarrollo de software (PPT 10)

Es una filosofía o enfoque más viejo que agile, que nace en Japón (‘40) con Toyota, y busca maximizar el valor generado al cliente con el mínimo consumo de recursos, argumentando que todo el esfuerzo que no cree valor, se considera desperdicio. Esto permite reducir los esfuerzos en crear artefactos que no agreguen valor a un producto, concentrando los esfuerzos en los aspectos esenciales. La filosofía “Lean”, conduce a una visión integrada de la cultura y la estrategia para atender al cliente final con alta calidad, bajo costo y tiempo de entrega, produciendo exactamente lo que el cliente final quiere, cuando lo quiere, dónde lo quiere, a un costo mínimo y precio justo. Siendo el cliente quien determina si el servicio o producto que la empresa entrega tiene valor o no. Para eso utiliza el método Just in time, que propone producir solo lo necesario, en la cantidad necesaria, en el momento necesario.

7 principios Lean

Estos principios apuntan al desarrollo de productos en la industria de manufactura (productos tangibles). Como el desarrollo de software es una actividad que produce productos intangibles, fue necesario adaptar dichos principios en el contexto del desarrollo de software. Empezaron siendo filosofías separadas pero se fueron mezclando con agile. En nada se contradicen. Define desperdicio como todo lo que no genera valor. Una interpretación es que los niveles intermedios, managers, de supervisión, tienen funciones necesarias ej apoyo, resolución de problemas, es importante no reducirlas del todo pero que estén el mínimo posible.

1) Eliminar desperdicios: El más importante. Debemos tener un proceso que sea eficiente y que cada uno de sus pasos sume valor, todo paso o parte que no contribuye a la generación de valor produce un desperdicio. Este principio se relaciona con los 3,7 y 10 de ágil (entregar software funcionando frecuentemente / el software funcionando es la principal medida de progreso / la simplicidad o maximizar el trabajo no realizado es esencial). **Evitar:**

- Producir funcionalidades en exceso que luego no se usan (recordar que el 80% del valor del producto está en el 20% de las funcionalidades, esto significa que no produzcamos funcionalidades que realmente no se necesiten ya que es un desperdicio y estaríamos malgastando recursos)
- Re-trabajo (significa volver hacer algo que debería haber salido bien en el primer intento, en esta filosofía se considera desperdicio ya que el cliente no paga más por los errores internos que corregimos)
- Que los artefactos se vuelvan OBSOLETOS antes de terminarlos

2) Amplificar el aprendizaje: Está relacionado con la transparencia y transformación del conocimiento implícito en explícito, lo cual fortalece el equipo de trabajo. Se quiere crear y mantener una cultura de mejoramiento continuo, donde los individuos intercambien sus experiencias y conocimientos para contribuir con el aprendizaje colectivo. Por lo tanto, todo

conocimiento que se genere debe compartirse para que sea fácilmente accedido por toda la organización. Este principio se relaciona con los 2 y 4 de ágil (recibir requerimientos aún en etapas finales / personas técnicas y no técnicas trabajando juntos en todo el proyecto)

3) Embeber la integridad conceptual: La calidad del producto no se negocia y la simplicidad es un factor clave. Significa construir un producto coherente, simple y de calidad, donde todas las partes encajan de manera armónica para satisfacer las necesidades del cliente. En Lean se logra no negociando la calidad del producto, evitando el desperdicio, y manteniendo siempre la visión global del producto.

- Se dice que la calidad del producto no se negocia porque se trata de un valor absoluto, no es una variable que pueda sacrificarse para ahorrar tiempo o costos.

4) Diferir compromisos hasta el último momento responsable: Postergar la toma de decisiones lo suficientemente hasta tener la mayor información necesaria para tomar esa decisión, sin que ese momento sea muy tarde como para perder oportunidades o poner en riesgo el proyecto (diferir la decisión irreversible). Se puede reflejar en Agile con el Product Backlog, donde nunca se tiene el 100% de los requerimientos, sino que se va completando a medida que se tiene más información sobre lo que se necesita implementar. Las mejores estrategias de diseño están basadas en dejar abiertas opciones de forma tal que las decisiones irreversibles se tomen lo más tarde posible.

5) Dar poder al equipo: Se le debe otorgar libertad de acción y poder de decisión al equipo. El equipo debe ser multifuncional y autogestionado, y sus miembros deben estar capacitados y motivados. Se relaciona con el principio 11 de ágil (las mejores arquitecturas, requisitos y diseños emergen de equipos autoorganizados). Implica no subordinar al equipo, ya que si los superiores interfieren de forma excesiva, se limita la capacidad intelectual del equipo y se debilita su autonomía, lo que genera desmotivación y reduce la creatividad. En cambio, al confiar en el equipo, se promueve la responsabilidad compartida, la toma de decisiones informadas y el compromiso con los resultados. Dar poder al equipo no significa ausencia de liderazgo, sino entrenar líderes capaces de guiar y delegar.

6) Ver el todo: Se busca tener una visión que permita asociar y comprender el todo, el producto y el valor agregado que hay detrás. En lugar de enfocarse en metas particulares, Lean promueve que todos los objetivos parciales estén alineados con los objetivos globales de la organización. Esto implica reconocer que mejorar un sector de manera aislada puede no generar un beneficio real si no contribuye al flujo general del valor. Es importante comprender el impacto de cada acción dentro del conjunto y colaborar de forma transversal para que el resultado final sea coherente, eficiente y verdaderamente valioso para el cliente.

7) Entregar lo antes posible: La idea es entregar al cliente software funcionando lo más rápido posible y que este producto sea útil para él. Se habla de entregarle software antes de que las necesidades de negocio cambien, porque el ambiente donde se desempeña cambia. Se relaciona con el principio 1 de ágil (satisfacer al cliente con entregas tempranas), Lean propone acotar ciclos de desarrollo y entregar rápidamente incrementos pequeños de valor, lo que permite salir pronto al mercado con un producto mínimo y valioso.

Los 7+1 desperdicios de Lean

1) Características extras: Ocurre cuando se desarrollan funcionalidades, componentes o detalles que el cliente no pidió o no necesita realmente. A primera vista puede parecer que agregar más características aporta valor, pero en realidad genera costos adicionales, complejidad innecesaria y re-trabajo futuro. Este problema no solo se da al desarrollar funcionalidades innecesarias, sino también al realizar Up front specification (especificación anticipada), es decir, cuando se definen requerimientos demasiado temprano, antes de tener la información suficiente. En las etapas iniciales, suele haber incertidumbre o falta de claridad sobre lo que realmente se necesita, por eso, si se detallan todos los requerimientos desde el principio, es probable que parte de esa información quede obsoleta, incompleta o incorrecta cuando llegue el momento de implementarla. Se relaciona directamente con el principio 4 Lean (diferir compromisos hasta el último momento responsable).

2) Trabajo a medias: Esto representa el trabajo que se realiza a medias (US, casos de uso, bugs, etc) sin terminar, lo cual genera que deba ser retomado posteriormente para su finalización. Este tipo de trabajo interrumpido o inconcluso genera acumulación, lo que retrasa la entrega de valor y obliga a retomar tareas más adelante, lo que implica pérdida de tiempo y esfuerzo adicional. Es por esta razón, que en metodologías ágiles se promueve la gestión binaria del trabajo, donde una tarea está “lista” o “no lista”, sin porcentajes de avance. Decir que una tarea está “80% terminada” no aporta nada de valor, por eso la única medida que importa en ágil y Lean es el trabajo terminado y entregado.

3) Proceso extra: Esto se refiere a los pasos extras que se ejecutan en el proceso de desarrollo que no aportan valor al producto. No destinar esfuerzo a actividades que no aportan valor. Los procesos definidos sobrecargan mucho a las personas con tareas que no aportan mucho valor, y como resistencia surgieron los procesos empíricos, que buscan eliminar todos aquellos pasos que no aportan valor al desarrollo del producto.

4) Búsqueda de información: Esto termina siendo un problema importante y difícil de manejar si no se implementa la disciplina SCM, porque no contamos con información suficiente y/o de fácil acceso para tener una trazabilidad sobre los ítems de configuración, lo que dificulta identificar el impacto de los cambios y ralentiza la resolución de errores.

5) Defectos: Son errores que se trasladan de una etapa a otra etapa posterior en la cual se produjo dicho error. Esto provoca retrabajo, debido a que Testing debe “devolver” la funcionalidad a desarrollo para la resolución de defectos o bugs. Una solución a este problema es la disciplina de Aseguramiento de Calidad, que permite evitar esos errores.

6) Esperas: Esto es el tiempo de espera que una persona tiene que pasar para poder realizar su trabajo. Porque depende del trabajo de otra persona o requiere aprobación de un superior. Son buenos los equipos multidisciplinarios evitando estos tiempos de espera.

7) Cambios de tareas: Cuando se intenta realizar varias tareas al mismo tiempo o se interrumpe continuamente una tarea para atender otra. Se necesita un tiempo de adaptación para recordar y retomar el contexto anterior. Relacionado con el enfoque Just in time, ya que promueve la fluidez y sincronización del trabajo. Se busca que cada tarea se realice en el momento justo y en la cantidad justa, sin acumular trabajo pendiente.

7+1) Talento no utilizado: No utilizar la experiencia, conocimiento y creatividad del personal, es el nuevo desperdicio que se agregó después. Por ejemplo hacer testing automatizado cuando la experiencia del tester podría generar mejores pruebas de testing, u otro ejemplo típico es cuando un desarrollador es tan bueno que quiere pasar a ser líder de equipo pero como las empresas no lo capacitan, pasaste de tener un buen desarrollador a un mal líder. Y ahí perdes talento por más que siga ejerciendo en tu empresa.

Relación Lean-Ágil: Lean es previo al Manifiesto Ágil. Algunos principios de Agile heredan de los fundamentos de Lean. Ambos están orientados al cliente, a proveer el máximo valor posible. Para esto, entienden que los individuos de los equipos deben estar motivados, de forma tal que la sinergia de este facilite el desarrollo del proyecto. Esta sinergia, se interpreta como un valor agregado para el cliente. La flexibilidad para adaptarse a los cambios y ofrecer valor al cliente, es también un denominador común de ambos enfoques. Otro aspecto que tienen en común es generar productos de calidad y de mejora continua

Método Kanban en el contexto del desarrollo de software (PPT 10)

Es un método para definir, gestionar y mejorar servicios que entregan trabajo del conocimiento, tales como servicios profesionales, trabajos o actividades en las que interviene la creatividad y el diseño tanto de productos de software como físicos.
NO es ni un proceso de desarrollo de software ni una metodología de administración de proyectos. Si un enfoque para la gestión de formas de trabajo (procesos) para obtener mejora continua. Esto lo logra a través del principio de “empieza por donde estés”, es decir, no plantea introducir cambios revolucionarios, sino que introduce mejoras graduales a un proceso de desarrollo de software o a una metodología de administración de proyectos ya existente en una organización. Esto permite reducir la resistencia al cambio por parte de las personas, fomentando la evolución gradual de los procesos existentes. No podes mejorar algo que no entendes, por lo que el primer paso siempre es entender bien el dominio.

Este enfoque surge con estudios de Toyota, para mejorar las técnicas de almacenamiento y tiempo de stockeo en supermercados. Para su aplicación en el desarrollo de software, al igual que Lean fue necesaria una adaptación. Kanban aprovecha los principios de Lean:

- Definir el valor desde la perspectiva del cliente, asegurando que cada tarea aporte un beneficio real
- Limitar el trabajo en proceso (WiP), para evitar la sobrecarga y mejorar la productividad
- Identificar y eliminar desperdicios, reduciendo actividades que no agregan valor
- Detectar y eliminar barreras en el flujo de trabajo, es decir, todo aquellos que retrase o interrumpa el avance
- Fomentar una cultura de mejora continua, promoviendo la reflexión constante y la adaptación progresiva del proceso

Kanban busca que los equipos trabajen de forma más eficiente, visualizando el flujo de tareas, limitando el trabajo simultáneo y promoviendo la mejora constante de procesos

6 prácticas de Kanban

1) Visualización del trabajo: El método Kanban utiliza un mecanismo de señalización para hacer visible el trabajo que es requerido por el cliente. Para ello, divide el trabajo en piezas de trabajo (que pueden ser U.S., Features, bugs, temas, épicas, cambios, etc.) y las escribe en tarjetas señalizadoras (kan-bans) que serán ubicadas en tableros kanban.
Estas tarjetas permiten indicar cuando el trabajo se puede "pullear" para realizar un trabajo determinado, además de señalar en qué parte del proceso se encuentra.
El tablero kanban representa un sistema de flujo en el que las piezas de trabajo fluyen a través de las diversas etapas de un proceso, de izquierda a derecha. Cada etapa es representada por una columna del tablero, sobre las cuales se aplica la teoría de colas.
Se logra **transparencia** al hacer visible para todo el equipo el trabajo mediante un tablero kanban siempre disponible y a la vista. Existen dos tipos de columnas:

- **Producción:** Piezas sobre las que se está trabajando.
- **Acumulación:** Piezas que están listas para pasar a la siguiente etapa (sistema de arrastre).

En conjunto, esta práctica facilita la gestión del flujo de trabajo, mejora la comunicación del equipo y promueve una toma de decisiones más ágil y basada en la realidad del proceso

2) Limitar el WiP (Work in Progress): Una de las prácticas esenciales de Kanban es establecer límites claros al trabajo en progreso (WiP), es decir, cuántas tareas pueden estar activas simultáneamente en cada etapa del flujo de trabajo. Este control permite evitar la sobrecarga del equipo y prevenir los cuellos de botella que pueden ralentizar el proceso.
Al limitar el WiP, se implementa un sistema de arrastre (pull system): el trabajo solo se incorpora a una nueva etapa cuando hay capacidad disponible, es decir, cuando una tarea anterior ha sido completada. Esto contrasta con los sistemas de "empuje", donde las tareas se trasladan al siguiente paso sin considerar la disponibilidad del equipo o del proceso siguiente. Tener demasiadas tareas abiertas al mismo tiempo genera ineficiencias, pérdida de foco y mayores tiempos de entrega. Por eso, observar, ajustar y optimizar continuamente la cantidad de trabajo en progreso resulta clave para alcanzar una mejor calidad del producto, reducción de los tiempos de entrega y aumento del flujo de valor hacia el cliente

3) Gestionar el flujo de trabajo: El trabajo se representa y gestiona a través de un tablero permanente, donde las tareas fluyen de manera continua a lo largo de las distintas etapas del proceso. A diferencia de otros enfoques, no existen iteraciones, proyectos o ciclos cerrados: Cada pieza de trabajo se gestiona de forma individual y se avanza según su propio ritmo. Este tablero puede ser compartido entre distintos equipos o proyectos, ya que su objetivo es reflejar el flujo global del trabajo y permitir una visión integral del proceso
El propósito es lograr un flujo constante y sin interrupciones, optimizando el paso de las tareas de una etapa a otra. Para ello, es necesario analizar de forma continua el comportamiento del flujo, identificando posibles cuellos de botella, recursos subutilizados, demoras, tiempos de entrega y tiempos de espera. Este análisis permite detectar oportunidades de mejora y garantizar un proceso más eficiente, predecible y equilibrado.
Es la principal diferencia, scrum habla de proyectos que inician y finalizan, a través de ese spring tenes que entregar ciertos requerimientos en un timebox especifico, en cambio en kanban solo se habla de flujos de trabajos. Va pasando por c/u de los pasos como la teoría de colas y vas entregando.

4) Hacer explícitas las políticas: Las políticas de proceso deben ser escasas, simples, estar bien definidas, visibles, deben aplicarse siempre, y tienen que ser fácilmente modificables. Es una buena práctica poder cambiar fácilmente las políticas, ya que si producen efectos contraproducentes para nuestro proceso o también se considera que no pueden aplicarse las mismas, deben poder cambiarse. También es importante visualizar las políticas; por ejemplo, colocando resúmenes entre las columnas donde se describe lo que debe estar hecho antes de que una tarjeta se mueva de una columna a la siguiente. También se suele colocar el WIP de cada columna. Las políticas de calidad o DoD deben estar definidas, publicadas y promovidas para que se cumplan y mejorar continuamente.

5) Mejorar y evolucionar: Kanban propone una cultura de mejora continua, donde no introduce cambios significativos en los procesos de desarrollo, sino que identifica ese proceso, lo visualiza (hacer explícito para todos los miembros) y propone mejoras sobre él, las cuales se van aplicando de forma gradual a lo largo del tiempo.

6) Circuito de retroalimentación: Son una parte esencial para cualquier proceso controlado que nos ayuda a realizar cambios evolutivos. Se debe definir con qué frecuencia se realizan las reuniones, donde depende en qué contexto se presentan ya que es importante para el resultado. Si se realizan revisiones demasiadas frecuentes pueden obligar a cambiar cosas que no se vieron con los cambios anteriores, pero si no son demasiadas frecuentes, existe un bajo rendimiento durante mucho tiempo.

Procedimiento para aplicar Kanban

1) Empezar con lo que se tiene: Kanban no busca reemplazar los procesos actuales, sino mejorarlos gradualmente. Paso 1 es entender el proceso existente, cómo fluye el trabajo, quién participa, qué etapas existen y cuáles son los principales problemas o demoras.

2) Identificar las unidades de trabajo: Se deben definir qué tipos de elementos se gestionan en el tablero Kanban. Pueden ser: US, casos de uso, defectos, mejoras, solicitudes del cliente, etc. Cada una de estas unidades representa una tarea o pieza de trabajo que avanza a través del flujo hasta completarse

3) Identificar las clases de servicio: No todo el trabajo se trata igual. Las clases de servicio permiten definir prioridades y políticas específicas para distintos tipos de tareas. Ej:

- **Requerimientos nuevos:** Siguen el flujo normal de desarrollo
- **Defectos críticos:** Tiene prioridad y se corrigen de inmediato
- **Solicitudes de mantenimiento:** Pueden tener un flujo o reglas distintas

Cada clase de servicio puede tener su propio criterio de aceptación (DoD) y tiempos de entrega

4) Visualizar el flujo de trabajo: Aquí se diseña el tablero Kanban, representando el proceso con columnas que muestran las distintas etapas del trabajo (por ejemplo: “por hacer”, “en progreso”, “en revisión”, “hecho”). El objetivo es que todo el equipo pueda ver el estado de cara tarea y detectar fácilmente dónde se acumula el trabajo o se detiene el flujo

5) Definir políticas para cada clase de servicio: Se deben establecer reglas claras sobre cómo se gestionará el trabajo. Estas políticas incluyen:

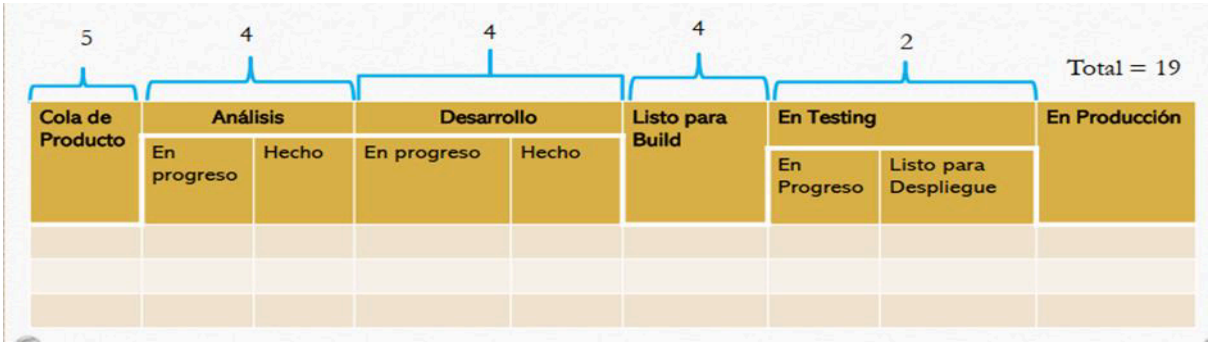
- Límites de trabajo en progreso (WiP) para evitar sobrecarga
- Colores en tarjetas para diferenciar clases de servicio
- Criterios de finalización
- Fechas de entrega
- Capacidad del equipo por columna o tipo de tarea

Deben ser visibles para todos, simples y modificables si se detecta que no funcionan bien

6) Identificar y resolver cuellos de botella: Una vez que el tablero está en uso, se debe observar cómo fluye el trabajo y detectar puntos donde se acumulan tareas o se retrasa el proceso. Los cuellos de botella pueden solucionarse de varias maneras:

- Aumentando recursos en esa etapa
- Redefiniendo los límites de WiP
- Mejorando la comunicación o automatizando tareas

El objetivo es lograr un flujo continuo y equilibrado, evitando bloqueos y desperdicios



9 Valores de Kanban

Pueden resumirse en una sola palabra: Respeto. Sin embargo, este concepto se desglosa en una serie de nueve valores fundamentales que guían la práctica y la cultura de trabajo:

1) Transparencia: Compartir la información de forma abierta y clara mejora el flujo de valor dentro del negocio. Implica usar un lenguaje sencillo y directo, hacer visible el trabajo y permitir que todos los miembros del equipo comprendan el estado real de los procesos.

2) Equilibrio: La efectividad de un sistema depende del equilibrio entre sus diferentes aspectos, como la demanda y la capacidad, los tiempos y los recursos, o las prioridades y los objetivos. Si este equilibrio se pierde durante un tiempo prolongado, el sistema colapsar o volverse ineficiente.

3) Colaboración: Trabajar juntos mejora los resultados. La colaboración es el núcleo del método, ya que permite que las personas compartan conocimientos, resuelvan problemas en conjunto y encuentren soluciones que beneficien al equipo y al flujo de trabajo.

4) Foco en el cliente: Todo sistema Kanban está orientado a entregar valor al cliente. El flujo de trabajo debe conducir hacia un resultado que satisfaga una necesidad o un pedido del cliente, ya sea interno o externo a la organización. Mantener este enfoque garantiza que las acciones del equipo estén alineadas con los objetivos reales del negocio.

5) Flujo: El flujo de trabajo representa cómo las tareas avanzan desde su inicio hasta su finalización. Visualizarlo y entenderlo es esencial para detectar bloqueos, cuellos de botella y oportunidades de mejora. Un flujo estable y continuo asegura una entrega de valor más predecible y eficiente.

6) Liderazgo: No depende de la jerarquía, sino de la capacidad de inspirar y guiar a los demás. Cualquier miembro del equipo puede ejercer liderazgo a través del ejemplo, la comunicación, y la reflexión, impulsando la mejora continua y la entrega de valor.

7) Entendimiento: La mejora comienza con el conocimiento de la situación actual, tanto a nivel individual como organizacional. Comprender los procesos, las fortalezas y las limitaciones permite avanzar de manera consciente hacia un estado mejorado.

8) Acuerdo: Kanban promueve un compromiso colectivo para avanzar hacia los objetivos comunes, respetando las diferencias de opinión y buscando puntos de encuentro. No se trata de lograr consenso absoluto, sino mantener un compromiso dinámico que permita mejorar de forma continua.

9) Respeto: El respeto es la base sobre la que se sustentan todos los demás valores. Implica valorar, comprender y considerar a las personas, sus capacidades, opiniones y contribuciones. Fomentar una cultura de respeto fortalece la confianza, la comunicación y el trabajo en equipo, pilares esenciales del método Kanban.

Principios directores

Los principios directos de Kanban son los pilares que guían su aplicación y evolución dentro de una organización. Estos principios orientan las decisiones y acciones del equipo para lograr un flujo de trabajo eficiente, adaptable y centrado en la mejora continua.

1) Sostenibilidad: Se refiere a mantener un ritmo de trabajo equilibrado y saludable a largo plazo. En Kanban, no se busca aumentar la productividad a cualquier costo, sino lograr un flujo de trabajo estable que el equipo pueda sostener sin agotamiento ni sobrecarga. También implica tener una cultura de mejora continua, donde el equipo se enfoca en optimizar gradualmente sus procesos, en lugar de aplicar cambios.

2) Orientación al servicio: Kanban pone el foco en entregar valor real al cliente, entendiendo que cada flujo de trabajo existe para ofrecer un servicio o satisfacer una necesidad. Este principio invita a ver el trabajo no como una lista de tareas internas, sino como un conjunto de servicios que deben cumplir expectativas, aportar valor y generar satisfacción. Orientarse al servicio implica medir el rendimiento no solo por la cantidad de tareas completadas, sino por la calidad de entrega y el impacto que genera en el cliente.

3) Supervivencia: Este principio está relacionado con la capacidad de adaptación y competitividad de la organización. En un entorno cambiante, las empresas y equipos deben ser capaces de responder rápidamente a nuevas demandas, tecnologías o contextos de mercado. Kanban fomenta esta adaptabilidad a través de su enfoque visual, la mejora continua y el control del flujo. De este modo, los equipos pueden detectar problemas, ajustar sus procesos y mantenerse competitivos frentes a los cambios.

6 Principios fundacionales

Los cuáles pueden ser divididos en dos grupos:

Principios de gestión al cambio: Cada organización está compuesta por una red de individuos conectados psicológica y socialmente, lo que naturalmente genera cierta resistencia al cambio. Kanban reconoce este aspecto humano y propone tres principios claves para gestionar el cambio de manera efectiva:

- **Empezar con lo que se está haciendo actualmente:** Este principio busca comprender los procesos existentes y respetar los roles, responsabilidades y flujos de trabajo actuales. De esta manera, se minimiza la resistencia al cambio, ya que se valoran las prácticas y la experiencia del equipo que las lleva a cabo. Además, los procesos vigentes, aunque imperfectos, contienen la sabiduría acumulada y el potencial de mejora necesarios para avanzar hacia un sistema más eficiente.
- **Acordar en buscar la mejora a través del cambio evolutivo:** Kanban promueve un enfoque de mejora gradual y constante, evitando imponer transformaciones drásticas que generen resistencia o desorganización. El objetivo es evolucionar progresivamente desde la situación actual hacia una más óptima, evaluando los avances y adaptando el proceso conforme se aprende.
- **Fomentar el liderazgo en todos los niveles:** El liderazgo no se limita a las posiciones jerárquicas, sino que debe promoverse en cada miembro. Esto implica incentivar la participación, la responsabilidad compartida y la capacidad de tomar decisiones informadas que impulsen la mejora continua en todos los niveles.

Principios de despliegue de servicios: Las organizaciones, sin importar su tamaño, funcionan como ecosistemas de servicios interdependientes. Kanban también aborda esta realidad con tres principios de entrega o despliegue de servicios:

- **Entender las necesidades y expectativas de los clientes y enfocarse en ellas:** El objetivo principal de Kanban es entregar valor al cliente. Para lograrlo, se deben comprender sus requerimientos y orientar todos los esfuerzos hacia satisfacer sus expectativas de manera efectiva y continua.
- **Gestionar el trabajo permitiendo la autoorganización:** Kanban fomenta la autonomía del equipo, permitiendo que las personas se organicen de forma natural en torno a las tareas. Esta autogestión promueve la responsabilidad, la eficiencia y la colaboración entre los miembros del equipo.
- **Evolucionar las políticas para mejorar los resultados hacia el cliente y el negocio:** Las políticas deben revisarse y ajustarse de manera continua, con el fin de adaptarse a las necesidades cambiantes y mejorar tanto la calidad del servicio como los resultados del negocio. Esta flexibilidad es esencial para mantener la competitividad y la satisfacción del cliente.

Estos principios están estrechamente relacionados con el valor central de orientación al servicio y con la entrega de valor al cliente. Kanban hace hincapié en que el foco no debe estar únicamente en la ejecución del trabajo, sino en el valor que dicho trabajo genera para los consumidores del servicio. En definitiva, el éxito del método reside en comprender al cliente, optimizar el flujo de trabajo y evolucionar continuamente las políticas para alcanzar la excelencia operativa y organizacional.

Diferencia	SCRUM	KANBAN
Tiempo	Las iteraciones son de tiempo fijo	El tiempo fijo es opcional. La cadencia puede variar. Pueden estar marcadas por la previsión de los eventos en lugar de tener un tiempo prefijado.
Métrica por defecto	Velocidad	Lead time (tiempo de entrega promedio)
Agregar trabajo	No se puede agregar alcance una vez comenzado el sprint	Siempre que haya capacidad disponible
Roles	Tres roles: PO, SM y equipo	No se prescriben roles
Permanencia del tablero	En cada sprint se limpia	El tablero es persistente

Métricas en los 3 enfoques (PPT 11)

Una métrica es un número que representa el grado o presencia de un determinado conjunto de atributos respecto de lo que se quiere medir (proceso, producto o proyecto), expresadas de tal forma que permitan una medición objetiva de la realidad. Por ejemplo, la escala: NS, S, B, MB, E no sirve. Lo que sí sirve es cuántos E hay, cuántos S hay, etc. Debe ser posible medir en términos de la definición de esta y del esfuerzo que se debe aplicar para medirla. Es decir que la métrica no es la escala sino la cantidad de cierta escala.
Medir tiene un costo: Requiere tiempo, esfuerzo y recursos para planificar, ejecutar y analizar la información. Por eso, antes de definir métricas, se debe hacer un análisis de costo-beneficio (¿vale la pena invertir en medir esto?). A veces no es necesario medir con extrema precisión; Además, una métrica no sirve si solo se calcula y se guarda en un informe; debe utilizarse para tomar decisiones o mejorar procesos, de lo contrario el esfuerzo invertido se desperdicia. En el sw, muchas características del producto son difíciles de medir directamente. Por ejemplo, la usabilidad de un sistema no se puede cuantificar de manera directa. En estos casos, se utilizan indicadores indirectos, datos que reflejan la característica que se quiere medir. Por ejemplo, si queremos medir cuánto se usa una funcionalidad, podemos contar la cantidad de clics en el botón que la activa. Medir sirve:

- Para controlar y supervisar el desarrollo del proyecto
- Para predecir al estimar proyectos de software
- Para evaluar, por ejemplo, tener una noción de los costos.
- Para mejorar el proceso, el proyecto o producto

3 dominios de métricas

1) Métricas de proceso: Son métricas estratégicas a nivel organizacional, orientadas a mejorar el proceso y tienen como objetivo generar indicadores que permitan mejorar los procesos a largo plazo. Su objetivo no es evaluar personas ni proyectos concretos, sino entender y mejorar el proceso de desarrollo de software a nivel organizacional.

Para esto, se recopilan datos de distintas actividades o productos, como por ejemplo la cantidad de defectos que surgieron en todos los productos desarrollados. Luego, esos datos se despersonalizan: no se identifican proyectos, equipos ni personas, sino que se promedian y se presentan como indicadores generales (es decir, los datos no se utilizan de forma individual, sino que unifica para utilizarlos como un todo). De esta manera, se obtiene una visión objetiva del proceso en su conjunto, que refleja cómo está funcionando la organización en términos de calidad, eficiencia o cualquier otro aspecto medible. Estas métricas permiten identificar que partes del proceso son efectivas y cuáles necesitan mejoras. Por ejemplo, si se detecta que ciertos tipos de defectos se repiten en varios productos, eso indica un problema en el proceso que debería corregirse. Los beneficios se ven a largo plazo, ya que la organización puede usar estos indicadores para aumentar su nivel de madurez en la gestión de procesos, mejorar la planificación, la calidad y la consistencia de los productos que desarrolla.

2) Métricas de proyecto: Están enfocadas a los recursos que se dedican al proyecto, como costos, esfuerzos, estimaciones y tiempo. Son responsabilidad del Líder de proyecto y permiten al equipo adaptar el desarrollo de los proyectos y de las actividades técnicas. Estas métricas son privadas de ese proyecto y solo son visibles para los involucrados en el mismo. Se utilizan para mejorar la planificación del desarrollo, generando ajustes que eviten retrasos, reduzcan riesgos potenciales, y por lo tanto, problemas. Además, se utilizan para evaluar la calidad de los productos en todo momento, y en caso de ser necesario, modificar el enfoque para mejorar la calidad, minimizando defectos, retrabajo y por ende el costo total del proyecto. Las métricas de proyecto se consolidan con el fin de crear métricas de procesos que sean públicas para la organización de software como un todo.

3) Métricas de producto: Están enfocadas en lo que se construye, son responsabilidad del equipo de desarrollo, testing y son particulares de ese producto. Se utilizan con propósitos técnicos y tienen como objetivo generar indicadores en tiempo real de la eficacia del análisis, el diseño, la estructura del código, la efectividad de los casos de prueba y la calidad del software a construir. Se deben controlar los artefactos resultantes del proceso de desarrollo (componentes y modelos) para poder garantizar:

- Que cumplan con los requerimientos del cliente y los de calidad
- Que estén libres de errores y que se realizaron bajo los procedimientos de calidad



Hoy en día no es recomendable medir en líneas de código. Defectos:

- **Cobertura:** Tratar de que sea la mayor posible para cubrir el 100% del análisis de los defectos del producto.
- Defectos por severidad: Cantidad de defectos por escala de severidad de cada defecto. 5 defectos graves.
- Densidad de defectos: Cuantos defectos se encuentran por historia de usuario.

Métricas en los 3 enfoques

Antes de analizar las métricas por enfoque vamos a ver las métricas **básicas** para un proyecto de software, mínimas e imprescindibles que uno tiene que utilizar si tiene la intención de desarrollar una cultura de medición, aunque están más ligadas al **tradicional** y están fuertemente relacionadas con la triple restricción de proyecto (costo, alcance, tiempo)

- **Tamaño del producto:** Asociada a métrica de producto.
- **Esfuerzo:** Asociada a métrica de proyecto.
- **Calendario:** Asociada a métrica de proyecto.
- **Defectos:** Asociada a métrica de producto.

1) Métricas en el enfoque tradicional: Están basadas en la gestión de proyectos definidos, y poseen una mayor cantidad de métricas en comparación a los otros enfoques. Son una disciplina transversal. Cuando se planifican los proyectos, allí se definen qué métricas se utilizarán, quiénes serán responsables de estas, cómo se calcularán, etc. Las principales son **esfuerzo, tiempo, costo y defectos.** No todas las métricas le interesan a todo el mundo, dependiendo del momento y rol es lo que interesa medir realmente. Los perfiles más técnicos apuntan a cuestiones relacionadas con el producto y esfuerzo (porque es donde los desarrolladores asumen compromiso) mientras que en ámbitos organizacionales es más interesante el costo y tiempo. Ejemplos:

- **Testers (perfil técnico):** Tienen interés por el esfuerzo y cantidad de defectos encontrados principalmente, por que es lo que más afecta en este rol
- **Líder de proyecto (un perfil más organizacional):** El enfoque de las métricas está más relacionado con el calendario, costos, plazos de tiempo a cumplir, etc por el contrato que se tiene con el cliente, y la relación entre esfuerzo y tiempo.

2) Métricas en ambientes ágiles: Mientras que en el tradicional se suelen medir aspectos como el cumplimiento de plazos, el uso de recursos o la cantidad de tareas realizadas, en ágil las métricas se centran en el producto y en el valor entregado al cliente, no en el proceso en sí. Esto se guía en el principio ágil “la mejor métrica de progreso es el software funcionando” ya que refleja justamente esa idea: lo que realmente demuestra avance no son los informes, los gráficos ni los porcentajes, sino tener un producto que funciona, usable y que genera valor real al cliente. Además las métricas en ágil no son universales ni extrapolables (es decir, sirven solamente para ese producto en específico, no puede usarse para otros). Cada equipo tiene su propio contexto, madurez y forma de trabajo, por lo que una métrica útil para un grupo no puede servir para otro. Por eso, las métricas deben servir al equipo, ayudándolo a mejorar su desempeño y flujo de trabajo, no a compararlo con otros ni a imponer controles externos. Las métricas sirven para un equipo y no son extrapolables por lo que dice el agilismo a otros proyectos o equipos. **3 métricas representativas:**

- **Métrica de velocidad:** Es la más importante del enfoque, mide la cantidad de puntos de historia que se realizaron en un Sprint y fueron aceptados por el Product Owner. Se calcula, no se estima. Suele representarse con un gráfico de barras para medir la estabilidad del equipo (se busca desarrollo sostenible). Estos gráficos son permanentes durante todo el proyecto y son visibles para todo el mundo.
- **Métrica de capacidad:** O velocidad estimada. Es una métrica de proyecto que mide el compromiso de un equipo para un determinado sprint, en horas de trabajo ideales o SP. Se utiliza para planificar, ya que permite definir cuántas US se van a tomar del Product Backlog para implementar por Sprint.

- **Running Tested Features (RTF- Funcionalidades probadas en ejecución):** Mide la cantidad de features testeadas que están funcionando, es decir, cuantas piezas de producto (historias de usuarios, casos de usos, requerimientos) se terminaron y están en ejecución. El problema de esta métrica es que no tiene en cuenta la complejidad de las piezas de producto que se implementan. Si la curva es constante (llana) o tiene una pendiente negativa, entonces indica la existencia de un problema, ya que las características no incrementan o disminuyen en el tiempo. Esta última situación se presenta cuando una funcionalidad del sistema deja de ser válida

3) Métricas de software en Lean (Kanban): Las métricas no se enfocan tanto en medir el producto final, sino en evaluar el comportamiento del proceso de trabajo. Esto se debe a que Kanban está diseñado para gestionar un flujo continuo de tareas, sin un inicio ni un final definidos como en un proyecto tradicional. El objetivo es introducir mejoras en un flujo de trabajo continuo (es decir, el foco de medición es el PROCESO). Por eso, las métricas se utilizan para entender cómo fluye el trabajo a lo largo del sistema y detectar oportunidades de mejora. En lugar de medir "cuánto trabajo se completó" en un tiempo dado, se mide cuánto tarda el trabajo en atravesar el sistema desde que se inicia hasta que se entrega.

- **Lead Time o Elapsed Time (Plazo de ejecución o tiempo transcurrido):** Métrica de vista o perspectiva del cliente. Mide desde el momento en que el cliente me pide algo (entra al Backlog) hasta que yo se lo entrego.
- **Cycle Time (tiempo de ciclo):** Mide el tiempo desde que el equipo comenzó a trabajar sobre una funcionalidad pedida, hasta que se lo entrega, eliminando el tiempo de espera en el Backlog. Por esto se la considera como una vista interna, que sirve al equipo de trabajo y no es tan relevante para el cliente. Es igual al Lead Time menos el tiempo que estuvo en el Backlog.
- **Touch time (tiempo de contacto):** Mide los tiempos en los cuales las personas están efectivamente trabajando sobre una unidad de trabajo (columnas de producción), sin tener en cuenta aquellos tiempos de espera (columnas de acumulación). En cada columna, por ejemplo, en la columna Test de la imagen, se tiene una columna de trabajo y otra de acumulación para poder realizar el concepto de sistema "pull" o de arrastre

$$\text{Touch Time} \leq \text{Cycle Time} \leq \text{Lead Time}$$

- **Eficiencia del ciclo de proceso:** Esta métrica mide la eficiencia del tiempo para entregar un trabajo, respecto al tiempo destinado a su ejecución. Se calcula como **Touch time / Lead time.** Mientras más cercano a 1 sea su valor, más eficiente es el proceso, ya que todo el tiempo se estuvo trabajando sobre alguna funcionalidad, y no hubo muchos momentos de espera en columnas de acumulación.

Ejemplo: Supongamos que una funcionalidad tarda 10 días en completarse desde que se inicia hasta que se entrega (Lead time = 10 días). Pero el equipo solo trabajó activamente 2 días sobre ella (Touch time = 2 días). Entonces haciendo 2/10, nos indica que el 80% del tiempo la tarea estuvo esperando (en cola, revisión, test, etc).



Unidad 3: Gestión de software como producto

Prácticas continuas

Cuando hablamos de gestionar el software como producto, nos referimos a mantenerlo en un estado evolutivo y mejorable de forma constante, no como algo que se desarrolla una vez y se entrega terminado.

Continuous Integration (CI): Es una práctica de desarrollo que busca evitar los problemas derivados de integrar grandes cantidades de código de forma tardía. Consiste en que los desarrolladores integren su código en un repositorio compartido varias veces al día, permitiendo que cada integración sea verificada automáticamente mediante pruebas. De esta manera, se pueden detectar y corregir errores de forma temprana, reduciendo el costo y el impacto de los fallos.
El objetivo principal es garantizar que el software pueda ser desplegado en cualquier momento, asegurando que el código compile correctamente y mantenga su nivel de calidad constante. Para lograrlo, cada desarrollador realiza pruebas unitarias en su entorno de trabajo (en lo posible automatizadas) aplicando la metodología TDD (desarrollo guiado por pruebas), en este enfoque se define el comportamiento esperado del código y luego se desarrolla la funcionalidad que permite que dichas pruebas se cumplan. Una vez que el desarrollador confirme que su componente funciona correctamente, lo sube al repositorio de integración. De esta forma, el producto se mantiene en un estado estable y en condiciones de pasar a las pruebas de aceptación de usuario sin inconvenientes, fomentando un flujo de trabajo más ágil, seguro y colaborativo

Continuous Delivery (CD): Es una práctica de desarrollo de software que busca que el producto esté siempre en condiciones de ser liberado a producción en cualquier momento. Esto significa que el software se construye, prueba y valida constantemente para asegurar que cada nueva versión sea completamente funcional y estable. A diferencia de CI, que se enfoca en integrar y probar el código con frecuencia, la entrega continua agrega un paso adicional: Automatizar las pruebas de aceptación y el proceso de despliegue, de modo que el sistema siempre esté listo para su lanzamiento. La decisión de desplegar recae en una persona o rol en específico, generalmente el Product Owner, evalúa si el momento es adecuado, esto permite equilibrar la agilidad técnica con las necesidades del negocio.
Para que la entrega continua sea posible, es fundamental contar con un proceso de integración continua previamente establecido, ya que los artefactos generados deben poder pasar a producción de manera rápida, sencilla y confiable. Esto se logra mediante despliegues automatizados, que reducen el tiempo y el riesgo de error.
El objetivo es que cada versión del producto esté siempre en un estado “entregable”, es decir, que el código compile correctamente, que las pruebas se ejecuten sin fallos y que el sistema pueda ser desplegado sin contratiempos.

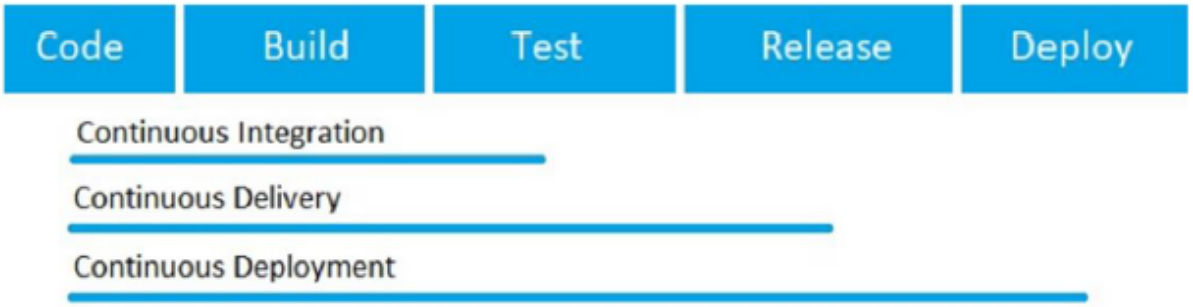
Continuous Deployment: Consiste en poner automáticamente en producción las nuevas versiones del software sin intervención humana. Es decir, cada vez que el código pasa todas las etapas del pipeline (complicación, pruebas unitarias, entre otras), se despliega

directamente al entorno de producción. Esto es solo posible si existe un pipeline de automatización muy sólido, que garantice que cada paso del proceso se realice en el orden correcto y con la verificación necesaria para evitar fallos. El objetivo principal es que las nuevas versiones lleguen al usuario final de forma rápida, confiable y transparente, sin que este perciba interrupciones o errores. Además, desplegar poco tiempo después de haber trabajado en el código tiene una gran ventaja: si surge un error, el desarrollador aún tiene fresco el contexto del cambio y puede corregirlo con mayor facilidad. Dentro del despliegue continuo existen distintas estrategias diseñadas para minimizar riesgos y asegurar una transición fluida entre versiones:

- **Canary Deployment:** Es una estrategia que consiste en liberar una nueva versión del software solo para un grupo reducido de usuarios antes de hacerlo para todos. Este pequeño grupo actúa como una especie de “canario en la mina”, permitiendo detectar posibles problemas sin afectar al resto de los usuarios. Si la nueva versión funciona correctamente y los usuarios no reportan fallos, se va ampliando progresivamente el porcentaje de usuarios que la reciben, hasta reemplazar por completo la versión anterior. Esta técnica tiene dos ventajas principales: permite observar cómo interactúan los usuarios reales con los cambios y limita el impacto de los errores, ya que cualquier falla afectará solo a una pequeña parte del sistema antes de ser corregida.
- **Blue-green deployment:** Es otra estrategia de despliegue seguro que utiliza dos entornos de producción paralelos:
    - El entorno azul (blue) contiene la versión actual del software, la que está en uso por lo usuario
    - El entorno verde (green) tiene la nueva versión lista para ser lanzada

Ambos entornos son prácticamente idénticos, pero solo uno recibe el tráfico de usuarios. Cuando la nueva versión (green) ha sido probada y validada, el tráfico se redirige gradualmente del entorno azul al verde. Si algo falla, el cambio puede revertirse de inmediato, volviendo a dirigir el tráfico al entorno azul. Una vez que la nueva versión está estable, el entorno azul se actualiza y queda preparado para el próximo ciclo de despliegue

La diferencia entre estas tres prácticas es el nivel de automatización de las pruebas.



Unidad 4: Aseguramiento de Calidad de Proceso y de Producto

Conceptos generales sobre calidad (PPT 12)

Calidad: La calidad no se limita solo a cumplir con los requerimientos explícitos, sino que también incluye aspectos implícitos que los usuarios esperan de manera natural al utilizar un producto o servicio. Formalmente, se puede definir como el cumplimiento de los requerimientos funcionales y de desempeño, así como de los estándares de desarrollo documentados, sumado a las características implícitas que se esperan de un software desarrollado profesionalmente. La calidad es todos los aspectos y características de un producto o servicio que se relacionan con su habilidad de alcanzar las necesidades.

Esto significa que un producto de calidad no solo hace lo que se le pidió de forma explícita, sino que también satisface expectativas no expresadas (facilidad de uso, estabilidad, rapidez, confiabilidad). La calidad es subjetiva y depende del contexto. Lo que para una persona puede considerarse un producto de alta calidad, para otra puede no serlo, dependiendo de sus necesidades, expectativas, entorno o circunstancias particulares. Por eso, la evaluación de la calidad no se limita a un estándar único, sino que combina la percepción del usuario con criterios técnicos y normativos

Gestión de calidad: La gestión de calidad del software para los sistema de software tiene tres intereses fundamentales:

- **A nivel de organización:** Busca establecer un marco general de procesos y estándares que conducirán a software de mejor calidad
- **A nivel del proceso:** Implica la aplicación de procesos específicos de calidad y la verificación de que continúen dichos procesos planeados
- **A nivel de proyecto:** Se ocupa también de establecer un plan de calidad para un proyecto

Aseguramiento de calidad (QA): En software se refiere a la definición y aplicación de procesos y estándares que buscan garantizar la obtención de productos de alta calidad. A diferencia del testing, que detecta errores una vez que el software ya está desarrollado, el aseguramiento de calidad tiene un enfoque **preventivo**, buscando que los problemas no lleguen a producirse. El equipo de QA debe ser **independiente** del equipo de desarrollo, de modo que pueda evaluar el software de manera imparcial y garantizar que se cumplan los estándares de calidad. Una parte clave del aseguramiento de calidad es la planeación de calidad, que consiste en desarrollar un plan específico para cada proyecto. Este plan define las cualidades que se esperan del software y cómo se medirán. Según Humphrey (1989), un plan de calidad efectivo puede estructurarse incluyendo varios elementos fundamentales:

- Introducción al proyecto
- Planes del producto (que indique las fechas de entregas críticas y las responsabilidades asociadas)
- Descripciones de los procesos que se aplicarán
- Metas de calidad (identificando y justificando los atributos esenciales que el producto debe cumplir)

- Gestión de riesgos (incluye la identificación de posibles problemas y estrategias para mitigarlos)

Aunque los estándares y procesos son importantes, los administradores de calidad deben enfocarse en desarrollar una “cultura de calidad” en la que todo responsable del desarrollo del software se comprometa a lograr un alto nivel de calidad del producto

Aseguramiento de calidad de proceso y de producto:



Problemas en la calidad: Un software de calidad no solo debería cumplir con los requerimientos, sino también que debería poder entregarse a tiempo, no exceder costos, etc

- Atrasos en las entregas (relacionado a calidad del proyecto)
- Costos excedidos (relacionado a calidad del proyecto)
- Trabajo fuera de hora (relacionado a calidad del proyecto)
- Requerimientos no claros (relacionado a calidad del producto)
- Software que no hace lo que debería (relacionado a calidad del producto)
- No se aplica SCM (relacionado a calidad del producto)

Calidad en el desarrollo de software: Los costos excedidos, retrasos en la entrega, falta de cumplimiento de los compromisos, requerimientos no claros hacen que la percepción de nuestro cliente sea mala. El proyecto es el medio que permite alcanzar el producto, por lo tanto, si el medio no tiene calidad, entonces difícilmente se pueda lograr un producto de calidad (proyecto es el medio para obtener un producto que se le entrega al cliente). Para que un SW sea de calidad debe satisfacer:

- Las expectativas del cliente, usuario y otros interesados (stakeholders)
- Las necesidades de la gerencia, del equipo de desarrollo y mantenimiento

Principios de calidad

- La calidad no se inyecta, debe estar embebida (es algo que se concibe desde el primer momento)
- Es una responsabilidad de todos los involucrados en el proyecto
- Las personas son clave para lograrla (se hace mucho hincapié en la capacitación, para brindar herramientas y conocimientos a los involucrado)
- Se debe liderar con el ejemplo
- No se puede controlar lo que no se mide, por lo tanto, es importante el uso de métricas

- Simplicidad
- Debe planificarse el aseguramiento de calidad
- El aumento de pruebas no aumenta la calidad, ya que esta se obtiene e inyecta a lo largo del proyecto, no al final (se debe comenzar con la calidad desde el inicio y no tratar de meterla al último)
- Debe ser razonable para el negocio
- Prevención es mejor que corrección, es menos costoso y corregir demanda un costo muy alto asociado al retrabajo

¿Por qué es importante el trabajo con calidad en la producción de software?:

- Es un aspecto competitivo, de supervivencia en el mercado. Las empresas certifican estándares de calidad (de procesos no de producto) para competir
- Retiene a los clientes e incrementa los beneficios. Siempre que se habla de calidad, en el fondo nos referimos al cumplimiento de las expectativas del cliente
- Determina un equilibrio costo-efectividad. La calidad reduce costos y retrabajos

Visiones de calidad (para quién es la calidad):

- **Visión del usuario:** Está relacionado a las expectativas que tiene el usuario para con el producto, y si este las satisface. Es la más complicada de establecer, porque están en la cabeza de los usuarios, razón por la cual el principio de comunicación constante es crucial, para ir validando en todo momento si estamos en el camino correcto. El agilismo trata de introducir la visión de calidad del usuario a través del rol de Product Owner (recordar que es el representante de los cliente y stakeholders)
- **Visión del producto:** Esto se asocial al nivel de satisfacción de los requerimientos particulares de cada producto
- **Visión del proceso:** Si el proceso de desarrollo utilizado es el correcto para el producto que se quiere desarrollar, es decir, que el proceso le aporta valor al producto y no produce desperdicios
- **Visión del valor:** Encontrar un equilibrio en la relación costo-beneficio, para obtener siempre el mayor valor posible para el cliente, y obviamente generar ganancias con el desarrollo del software
- **Visión trascendental:** Es una visión utópica, ya que se asocia a objetivos difíciles de alcanzar. Son objetivos que motivan a seguir en busca de la mejora continua, un ejemplo sería un producto con 0 defectos, difícilmente se consiga dicho objetivo, pero nos motiva a obtener un producto con la menor cantidad de defectos posibles

En el caso del software, la calidad subjetiva se relaciona principalmente con las características no funcionales, como la fiabilidad, el rendimiento y la usabilidad (el usuario no dice explícitamente esto pero internamente lo quiere y asume). Estas características reflejan la experiencia práctica del usuario; si el software no cumple con la funcionalidad esperada, los usuarios buscarán alternativas para lograr sus objetivos, sin embargo, si el sistema es poco fiable o demasiado lento, resulta prácticamente imposible que cumpla sus metas. Con frecuencia, los problemas de calidad no se deben únicamente a una mala gestión, procesos inadecuados o falta de capacitación, sino a presiones externas del negocio. Esto puede derivar en software con funcionalidades limitadas o con menor fiabilidad y rendimiento, afectando directamente la calidad del producto.

Calidad en el software

La calidad de software se asegura mediante la definición y ejecución de procesos organizados que guían la creación del producto. Generalmente, las organizaciones se apoyan en modelos de referencia, como la ISO 9001, que establece estándares y buenas prácticas para el desarrollo de software. Estos modelos permiten tanto implementar procesos de mejora continua como evaluar el grado de adherencia del proceso real a lo que se recomienda. Hay modelos para crearlos, mejorarlos, evaluarlos y medirlos. Los procesos, sin embargo, se instancian en los proyectos, es decir, se adaptan a la ejecución concreta de cada desarrollo. Son guías sobre cómo hacer las cosas, pero es en el proyecto donde se insertan las actividades necesarias para asegurar que lo que se está haciendo cumple con lo planificado y con los estándares de calidad deseados.



Dentro de estas actividades se incluyen revisiones técnicas, realizadas entre pares, cuyo objetivo es evaluar el producto y no a la persona, y pueden aplicarse en cualquier artefacto, como requerimientos, diseños o código. También existen auditorías, llevadas a cabo por personas externas, que validan que los procesos y productos cumplan con los estándares establecidos, aunque en metodologías ágiles se prefieren identificar y corregir los problemas de manera interna.

El producto se somete a evaluación a lo largo de todo el proyecto mediante diversas técnicas: revisiones para detectar defectos tempranamente, auditorías de configuración funcional o física para validar versiones de la línea base, y testing para controlar la calidad del producto ya desarrollado.

Calidad en el producto

Es importante que los requerimientos sean medibles (verificables) y objetivos (no ambiguos) para que estos nos sirvan a la hora de validar. La gestión de calidad en productos son acciones sistemáticas para lograr calidad, las cuales requieren un equilibrio:

- **Calidad programada:** Los alcances del producto planificado
- **Calidad realizada:** Lo que realmente se ha desarrollado del producto
- **Calidad necesaria:** Mínimas características que el producto debe tener, para que este satisfaga los requerimientos

Para obtener calidad en el producto se debe definir un proceso que debe ser conocido por todos los involucrados en el equipo, donde además de tener tareas técnicas (requisitos, análisis, diseño, etc) se incorporan disciplinas transversales como SCM, QA, planificación de proyectos y su seguimiento

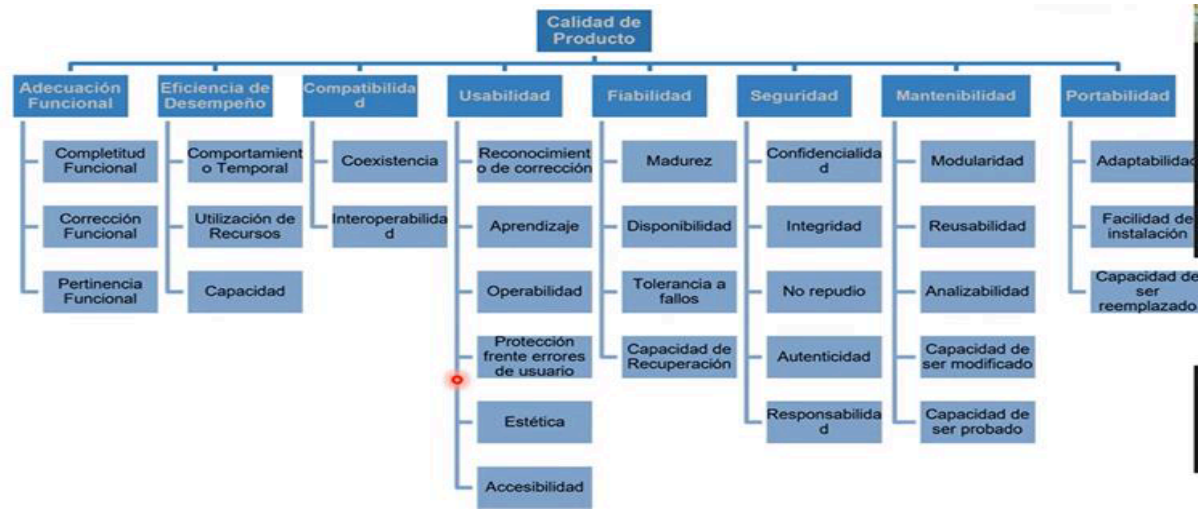
Modelos de calidad de producto

Al variar los requerimientos para cada producto en particular, no existen modelos de calidad que acrediten para un determinado software qué nivel de calidad posee. Si existen modelos como el Modelo de Barbacci, Calidad de Software (MCCALL) o la ISO 25.010 que permite medir epifenómenos como los RNF del software, para certificar calidad en ciertos aspectos

- Calidad en Uso (ISO 25010: 2011)

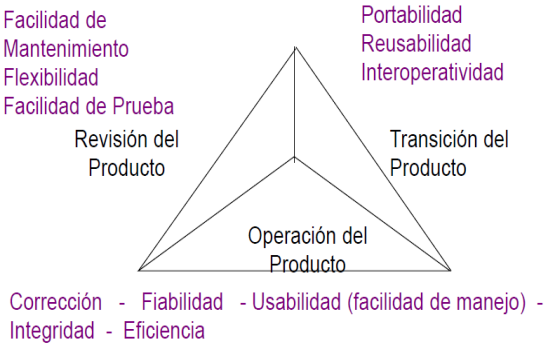
- Calidad de producto (ISO 25010:2011)
- Calidad de Datos (ISO 25012:2008)

Ejemplos: **ISO 25000 RNF:**

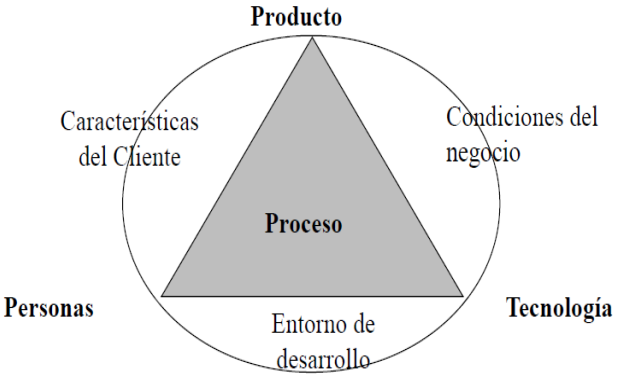


Modelo de Barbacci / SEI: 3 ejes con performance, seguridad y confiabilidad.

Calidad del SW Mccall: Tiene tres grandes agrupadores unos relacionados a la capacidad de verificación del producto (revisión del producto), otras con el producto en uso (operación del producto) y otras con los factores que influyen que el producto pueda crecer en el tiempo (transición del producto).



Calidad del proceso



No existe en la realidad un proceso que sirva para todas las organizaciones y proyectos en general. La calidad de proceso nunca es un fin en sí mismo, lo que realmente nos interesa es la calidad de producto.
Proceso con calidad → Producto con calidad.
Se insiste tanto en la calidad del proceso, debido a que es el único factor controlable para mejorar la calidad del sw. El avance de las tecnologías que se utilizan para construirlo es ajeno a la organización y no es posible controlarlo. Las características y necesidades del cliente tampoco se pueden modificar. Las personas involucradas en el proceso de desarrollo son difíciles de controlar también. Se aplica calidad en el producto y en el proceso. No se habla de aseguramiento de calidad en el proyecto, dado que esta se encuentra implícita por el hecho de que el proyecto implementa un proceso. Para garantizar la calidad se realizan auditorías. La calidad del producto se realiza mediante actividades de revisiones técnicas y de auditorías. Son las herramientas principales para el seguimiento. Para obtener calidad en el producto se debe definir un proceso que debe ser conocido por todos los involucrados en el equipo, donde además de tener tareas técnicas (requisitos,

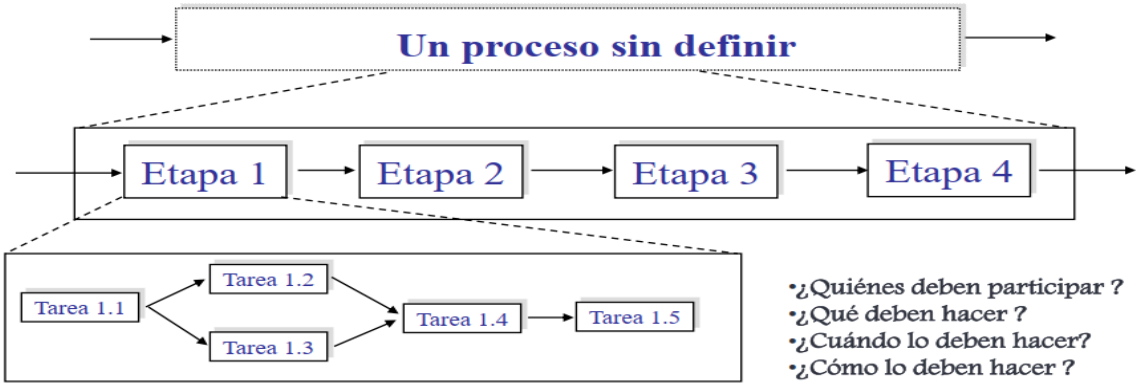
análisis, diseño, etc.). Se incorporan disciplinas transversales como SCM, QA, Planificación de Proyectos y su Seguimiento. **Objetivos de QA:**

- Realizar los controles apropiados del software y de su proceso de desarrollo.
- Asegurar el cumplimiento de los estándares y procedimientos para el software y el proceso.
- Asegurar que los defectos en el producto, proceso o estándares son informados a la gerencia para que puedan ser solucionados.

2 estándares de gestión de calidad del software:
Estándares de Producto: Definen las características que todos los componentes del producto deberían exhibir. Por ejemplo: estilos/buenas prácticas de programación.
Estándares de Proceso: Establecen los procesos que deben seguirse durante el desarrollo, por ejemplo: incluyen definiciones de requerimientos, diseño, validación, etc. Definen como deben ser implementados los procesos de software

Procesos

Lo primero que se debe realizar en una organización para lograr tener un proceso de calidad, es definirlo de forma explícita y comunicarlo a todo el equipo, para que así todos tengan una base y el conocimiento de la forma de trabajar. Se deben definir aspectos como etapas, subetapas, roles, qué se debe hacer, en qué momentos y cómo se deben hacer, etc.



Dentro de esa definición, la cual plantea las etapas para construir la parte técnica (Ingeniería de Requerimientos, Análisis, Diseño, Implementación, Prueba y Despliegue), también se deben incorporar disciplinas transversales a ellas, como la Planificación y Seguimiento de Proyectos, SCM y QA, independientemente de la metodología adoptada (tradicional o empírica). El proceso definido y adoptado, debe aportar valor al producto (es posible utilizar modelos de mejora de proceso como Kanban) y utilizarlos en los distintos proyectos de forma ADAPTADA. La adaptación de los procesos se da en aspectos negociables del mismo. Por ejemplo, el realizar Testing no es algo negociable.

- **Procesos definidos:** Se considera que el proceso es el único factor controlable, se asume que la mejora de la calidad continua se realiza sobre el proceso. Si el proceso cuenta con calidad, entonces el producto será de calidad. Los modelos de calidad están basados en procesos definidos, por lo que asumen que la calidad del producto se obtiene si se tiene un proceso de calidad para construir el producto.
- **Procesos empíricos:** Están basados en la experiencia, por lo que no consideran que la calidad en el proceso determine la calidad en el producto. Por otra parte, no

están de acuerdo con las auditorias, ya que asumen que los equipos son autoorganizados. Sin embargo, la calidad en estos procesos se lleva a cabo mediante revisiones técnicas y la constante inspección y adaptación del trabajo. Según los empíricos, siempre que las personas hagan lo que tengan que hacer el producto tendrá calidad

Actividades relacionadas con el Aseguramiento de la Calidad del Software.

Administración de la calidad de software: Busca asegurar que se alcancen los niveles requeridos de calidad para el producto de software, definiendo estándares y procesos de calidad apropiados (que deben ser respetados por todos). El objetivo es generar una cultura de calidad y que esta sea vista como una responsabilidad de todos en el equipo.
La idea es insertar en las actividades acciones tendientes a detectar lo más temprano posible oportunidades de mejora sobre el producto y sobre el proceso. Hay que tener ciertas condiciones respecto al GAC (reporte del grupo de aseguramiento de calidad):

- No debería reportar la gente que hace calidad al mismo gerente que reporta los proyectos (porque le quita independencia y libertad)
- El grupo de aseguramiento de calidad debería depender del gerente general
- Cuando sea posible, el grupo de aseguramiento de calidad debería reportar a alguien realmente interesado en el aseguramiento de calidad

Entre las 3 actividades que desempeña la administración de calidad, se encuentran:
1) Aseguramiento de calidad: Se encarga de definir estándares, procesos, procedimientos y modelos de calidad sobre los cuáles se van a realizar las comparaciones:

- Prácticas de aseguramiento de calidad
- Evaluación de la planificación del proyecto de sw, req y del proceso de diseño
- Evaluación de las prácticas de programación y prueba del software
- Evaluación de los procesos de planificación, integración y control de proyectos
- Adaptación de los procedimientos de calidad para cada proyecto

2) Planificación de calidad: Se debe planificar la calidad, qué actividades se van a llevar a cabo y cuándo. Se seleccionan los estándares aplicables para nuestro proyecto en particular y se los modifica si fuera necesario. Abarca determinar los productos de software que se desea tengan calidad y determinar sus atributos de calidad más significativos

3) Control de calidad: Es la ejecución de lo planificado, y se revisa en qué situación está el proyecto. Asegura que los procedimientos y estándares son respetados por el equipo

- **Revisiones de calidad:** Principal método de validación de la calidad de un proceso o un producto. El grupo examina parte de un proceso o producto junto a la documentación para encontrar potenciales problemas. Ej de varios tipos:
    - Inspecciones para remoción de defectos (producto)
    - Revisiones para evaluación de progreso (producto y proceso)
    - Revisiones de calidad (producto y estándares)
- Evaluaciones de software automáticas y mediciones

Lineamientos para la implementación de modelos de calidad en las organizaciones

2 Modelos para la mejora de procesos (IDEAL y SPICE):

La mejora continua de procesos significa comprender los procesos existentes y cambiarlos para incrementar la calidad del producto o reducir los costos y el tiempo de desarrollo. Los procesos definidos están de acuerdo con esta definición, ya que consideran que la calidad del producto final depende de la calidad del proceso (a diferencia de los empíricos que considera que los equipos son autogestionados). Los modelos son descriptivos, por lo tanto, no indican cómo hacer las cosas, el propósito de los modelos de mejora es analizar el proceso que tiene la organización y armar un proyecto cuyo resultado, en lugar de ser un producto, es un proceso mejorado que se vuelca de nuevo a la organización con la idea de que se produzca un producto mejor. Vamos a ver dos, el modelo IDEAL y el modelo SPICE:

Modelo IDEAL: Sirve como guía para planificar y ejecutar proyectos de mejora de procesos. Su objetivo principal es ayudar a definir, evaluar y optimizar los procesos existentes para alcanzar un mejor nivel de eficiencia y calidad. Es cíclico, una vez completa un ciclo de mejora, los resultados se analizan y se utilizan como base para iniciar un nuevo ciclo, generando así una mejora continua. En la fase de inicio, se busca generar el compromiso organizacional necesario para llevar adelante el proyecto. Esto incluye obtener un **sponsor**, que brinde apoyo (tanto económico como institucional) para impulsar el cambio. Esto es fundamental porque los proyectos de mejora de procesos rara vez son vistos como urgentes o prioritarios frente a las tareas operativas del día a día. Sin un respaldo formal, suelen quedar relegados o abandonarse antes de completarse. Asegurado el apoyo, el siguiente paso es **analizar** la situación actual de la organización y definir a dónde se quiere llegar. A este análisis se lo conoce como análisis de brecha y consiste en identificar las diferencias entre el estado actual de los procesos y el estado deseado o ideal. E un modelo de mejora de procesos que sirve como guía para inicializar, planificar e implementar acciones de mejora.

- **Inicialización:** Se reconocen las necesidades de cambio en la organización, razones para iniciar, determinar metas buscadas al proponer un cambio en el proceso. Se requiere que la organización apoye esta decisión de mejora (sponsoreo)
- **Diagnóstico:** Establecer la madurez actual de la organización y los riesgos asociados al proceso de mejora (revisar estado actual y futuro de la organización)
- **Establecimiento:** Durante la fase se elabora un plan detallado con acciones específicas, entregables y responsabilidades para el programa de mejora basado en los resultados del diagnóstico, y en los objetivos que se quieren alcanzar. Para elaborar el plan se parte de definir las prioridades para el esfuerzo de mejora
- **Ejecutar/Acción:** Efectuar los cambios y reunir información para aprender de la mejora. Se implementan las acciones planeadas en un proyecto piloto (no en todos los procesos, ya que es una prueba). Se implanta si la solución es satisfactoria.
- **Aprendizaje:** Se revisa toda la información recolectada en los pasos anteriores y se evalúan los logros y objetivos alcanzados para lograr implementar el cambio de manera más efectiva y eficiente en el futuro. Extrapolar la mejora al resto de proyectos en caso de que sea exitosa la ejecución o realizar correcciones en caso de que fracase el proyecto piloto.

Modelo SPICE (Evaluación de la capacidad de mejora de procesos de software): Es un modelo creado para la mejora de procesos software (ISO 15504). Es una adaptación para la evaluación de procesos de desarrollo software por niveles de madurez, dividido en 6 niveles. Es un modelo dual, teórico. Tiene 2 partes:

- Modelo de Calidad
- SPICE + IDEAL: Son ambos modelos de mejora de proceso. Estos modelos de mejora se usan para crear Proyectos de Mejora para una organización. El resultado de este proyecto va a ser un proceso definido que se usará para hacer Proyectos.

Este modelo establece conjuntos predefinidos de procesos con objeto de definir un camino de mejora para una organización. Establece 6 niveles de madurez para clasificar:

Nivel	Estado
Nivel 0 - Organización inmadura	La organización no tiene una implementación efectiva de los procesos
Nivel 1 - Organización básica	La organización implementa y alcanza los objetivos de los procesos
Nivel 2 - Organización gestionada	La organización gestiona los procesos y los productos de trabajo se establecen, controlan y mantienen
Nivel 3 - Organización establecida	La organización utiliza procesos adaptados basados en estándares
Nivel 4 - Organización predecible	La organización gestiona cuantitativamente los procesos
Nivel 5 - Organización optimizando	La organización mejora continuamente los procesos para cumplir los objetivos de negocio

Modelo de calidad para evaluar procesos (CMMI)

Es una guía teórica que define cómo debería desarrollarse el software dentro de una organización para asegurar que los resultados sean de calidad. Este modelo describe los lineamientos generales del proceso, como la definición de roles, la asignación de responsabilidad y las actividades que deben realizarse, con distintos niveles de detalle según las necesidades. Una vez definido ese proceso “ideal”, se compara con la realidad, es decir, con la forma en que se ejecutan los proyectos dentro de la organización. Esta comparación permite medir que tan alineado está el trabajo real con lo que plantea el modelo, y se realiza mediante auditorías de proyecto. No es lo mismo que un estándar de proceso, un modelo de calidad tiene como propósito evaluar y mejorar la madurez del proceso de desarrollo. Su objetivo principal es acreditar que una organización, o un área de ella, posee un cierto nivel de madurez en la forma en que desarrolla software, permitiendo identificar oportunidades de mejora y aumentar la eficiencia y calidad del producto

CMMI (Integración de Modelos de Madurez de Capacidades): Es un modelo de calidad para la mejora y evaluación de procesos para el desarrollo, mantenimiento y operación de sistemas de software. Su propósito es ofrecer un marco de buenas prácticas que las organizaciones puedan seguir para lograr resultados más predecibles, eficientes y de alta calidad. CMMI funciona como un modelo de referencia, lo que significa que describe los objetivos que deben alcanzarse, pero deja en manos de cada organización la elección de las prácticas o métodos específicos para lograr esos objetivos.
Este modelo se basa en la experiencia empírica: fue considerado a partir del análisis de prácticas exitosas utilizadas por distintas organizaciones a lo largo del tiempo, recopilando aquello que realmente funcionó en proyectos de software y procesos de gestión. De este modo, CMMI sirve como guía que ayuda a las organizaciones a mejorar progresivamente su madurez y capacidad de procesos, pudiendo aplicarse a un proyecto puntual, a un área específica o toda la organización.

La acreditación o certificación en CMMI se obtiene a través de una evaluación formal que combina un enfoque objetivo, mediante la comparación del proceso real de la organización con un modelo de evaluación estándar llamado SCAMPI (Standard CMMI Appraisal Method for Process Improvement), y un enfoque subjetivo, que toma en cuenta la experiencia y el juicio del evaluador sobre cómo se aplican los procesos. CMMI se estructura en tres constelaciones, que representan distintos ámbitos de aplicación del modelo:

- **CMMI-DEV (Desarrollo):** Orientado a los procesos de desarrollo de software. Proporciona guías para planificar, medir y mejorar los procesos de desarrollo. Posee dos formas de representación: por etapas (niveles de madurez) y continua (niveles de capacidad por proceso).
- **CMMI-ACQ (Adquisición):** Enfocado en organizaciones que adquieren productos o servicios desarrollados por terceros. Ayuda a gestionar la relación con los proveedores y a asegurar la calidad de los productos o servicios adquiridos, sin que la organización los produzca directamente.
- **CMMI-SVC (Servicios):** Dirigido a la gestión y mejora de los procesos de prestación de servicios, tanto internos como externos, garantizando que se cumplan los niveles de servicio acordados con los clientes.

Cada constelación incluye modelos, guías, capacitaciones y herramientas necesarias para que las empresas puedan implementar las prácticas recomendadas y, eventualmente, obtener la certificación que acredite su nivel de madurez organizacional, demostrando que sus procesos son confiables, controlados y orientados a la mejora continua.

Áreas de proceso: Un área de proceso es un conjunto de prácticas relacionadas en una zona que, cuando se implementan en conjunto, satisfacen un conjunto de objetivos importantes para hacer mejoras significativas en el mismo proceso. Existen 22 áreas de procesos en todos los niveles de madurez, son iguales en ambas representaciones, 16 de esas 22 áreas de procesos son comunes a todas las constelaciones CMMI.

Representaciones CMMI-DEV: El modelo CMMI-DEV se puede aplicar de dos maneras:
1) Por etapas: Esta representación proviene del modelo anterior, llamado CMM, y se centra en medir el nivel de madurez organizacional de una empresa o área de trabajo. La idea es clasificar a las organizaciones según su nivel de madurez que va del 1 al 5. Las organizaciones maduras (niveles 2 a 5) tienen procesos bien definidos, controlados y mejorados continuamente, mientras que las inmaduras (nivel 1) carecen de estabilidad y dependen demasiado del esfuerzo individual. Esta representación permite comparar fácilmente el nivel de madurez entre distintas organizaciones, para alcanzar un nivel superior, primero hay que cumplir con los requisitos de los niveles anteriores.

- **Nivel 1-Inicial:** La organización no tiene procesos estables. Cada proyecto se gestiona de forma improvisada, sin una planificación adecuada. Aunque algunas prácticas técnicas sean correctas, los resultados suelen ser impredecibles; hay retrasos, sobrecostos y fracasos frecuentes. Depende más del esfuerzo individual.
- **Nivel 2-Administrado:** La organización ya gestiona los requisitos de sus proyectos y planifica, mide y controla sus procesos. Existen seguimientos de calidad razonables y se trabaja con mayor orden. No dependen tanto de la improvisación.
- **Nivel 3-Definido:** Los procesos están bien documentados y estandarizados en normas, procedimientos, herramientas y métodos. Se capacita al personal, se aplican técnicas de ingeniería más rigurosas y se incorporan métricas y revisiones entre pares para asegurar calidad del trabajo

- **Nivel 4-Cuantitativamente administrado:** En este punto la organización utiliza métricas y datos estadísticos para medir la calidad y la productividad. Se toman decisiones basadas en datos y se gestionan los riesgos con información correcta. Los resultados son más predecibles y el software que se produce es de alta calidad
- **Nivel 5-Optimizado:** La organización alcanza la mejora continua. Utiliza innovaciones tecnológicas y mejoras de procesos para seguir perfeccionando su rendimiento. Se establecen objetivos cuantitativos de mejora, que se revisan constantemente según las metas del negocio. La organización aprende de sus errores y adapta sus procesos para ser cada vez más eficientes

2) Continua: La representación continua del modelo CMMI-DEV se enfoca en evaluar y mejorar la capacidad de cada proceso de manera **individual**, en lugar de medir la madurez global de toda la organización. Permite medir qué tan bien se ejecuta un proceso en particular. Su objetivo es acreditar la capacidad de la organización en cada área de proceso específica, identificando cuáles funcionan correctamente y cuáles necesitan mejora. Se puede concentrar sus esfuerzos en los procesos prioritarios, sin tener que modificar toda su estructura. Los procesos se evalúan según su nivel de capacidad que van del 0 al 5.

- **Nivel 1:** El proceso no se ejecuta o se realiza de forma incompleta.
- **Nivel 2:** El proceso está planificado y controlado.
- **Nivel 3:** El proceso se encuentra documentado y estandarizado.
- **Nivel 4:** Se gestiona con métricas cuantitativas que permiten analizar su desempeño
- **Nivel 5:** El proceso se optimiza de forma continua mediante innovaciones y mejoras tecnológicas. (Tiene 6 niveles en total ya que arranca desde cero)

Relación CMMI con ágil

Puede entenderse como un intento de combinar dos enfoques distintos para lograr la mejora de procesos y la calidad del software. Mientras que CMMI busca la madurez organizacional a través de procesos definidos, medibles y evaluables, ágil se centra en la flexibilidad, la adaptación al cambio y la entrega continua de valor al cliente. En los primeros niveles del CMMI (especialmente en nivel 1 y 2), es posible aplicar prácticas ágiles sin generar grandes conflictos. Identificar el alcance del trabajo, realizar el trabajo, gestionar los requerimientos, asignar responsabilidades, capacitar al equipo son actividades que encajan bien con ambos enfoques. Pero las diferencias comienzan cuando CMMI exige monitoreo objetivo y auditorías externas, ya que en metodologías ágiles el control y la evaluación suelen ser internos y colaborativos. Para que ambos enfoques puedan convivir, deben ceder parcialmente. Ágil debe aceptar ciertos mecanismos de control y trazabilidad (registrar requerimientos de forma trazable o mantener documentación mínima), mientras que CMMI puede flexibilizar la exigencia de auditorías formales y aceptar evidencias más ligeras.
A partir del nivel 3 y 4, las diferencias se hacen más marcadas. CMMI requiere procesos definidos y métricas cuantitativas para medir el rendimiento de los procesos y los productos, mientras que ágil no evalúa si se ha seguido un proceso específico, sino si el resultado aporta valor al cliente. CMMI busca predecibilidad y estabilidad, mientras que ágil se centra en la adaptación y la mejora continua basada en la experiencia.

- CMMI enfatiza la medición, los procesos y la disciplina organizacional, con un enfoque en la mejora continua del proceso y calidad del producto
- Ágil prioriza la respuesta rápida al cliente, la comunicación directa, la simplicidad y la colaboración del equipo

En cuanto a las características principales, CMMI busca una mejora organizacional, basada en la estandarización y la uniformidad de procesos, mientras que ágil promueve la mejora a nivel de proyecto, a través de la innovación, flexibilidad y aprendizaje práctico. CMMI desalienta los atajos, ya que se enfoca en cumplir cada etapa del proceso, mientras que ágil los alienta cuando ayuda a entregar valor más rápido. En relación con el cliente y la comunicación, CMMI se apoya en la infraestructura del proceso y en comités organizacionales, mientras que ágil pone énfasis en las personas y en el software funcionando como prueba de confianza, respecto al enfoque de trabajo, CMMI es descriptivo y cuantitativo, basado en datos y medidas numéricas, con una gestión de riesgos proactiva, mientras que ágil es prescriptivo y cualitativo, orientado a la acción directa, y su gestión de riesgos suele ser más reactiva, adaptándose sobre la marcha.

CMMI	MÉTODOS ÁGILES
➔ Mejora Organizacionalmente -Uniformidad -Nivelación	➔ Mejora en el Proyecto - Tradición Oral - Innovación
➔ Capacidad/Madurez - Éxito por Predictibilidad	➔ Capacidad/Madurez - Éxito por darse cuenta de oportunidades
➔ Cuerpo de Conocimiento - Cruzando dimensiones - Estandarizado	➔ Cuerpo de Conocimiento - Personal - Evolucionando - Temporal
➔ Reglas de Atajo - Desalentadas	➔ Reglas de Atajo - Alentadas

CMMI	MÉTODOS ÁGILES
➔ Comités	➔ Individuos
➔ Confianza del Cliente - En la Infraestructura del Proceso	➔ Confianza del Cliente - Sw funcionando, Participantes
➔ Cargado al frente - Mover a la derecha	➔ Conducido por Pruebas - Mover a la izquierda
➔ Alcance de la vista [Involucrado, Producto] - Amplio - Inclusivo - Organziacional	➔ Alcance de la vista [Involucrado, Producto] - Pequeño - Focalizado
➔ Nivel de Discusión - Palabras - Definiciones - Duradero - Exhaustivo	➔ Nivel de Discusión - Trabajo en mano

Aunque podemos observar similitudes también:

- Meta: Organizaciones de alto desempeño
- Ambas planean
- Ambas son CMMs (Consultant Money Makers)
- Ambas tienen reglas [Reglas = Requerimientos del proceso]
    - La violación tiene serias repercusiones
    - ‘SEPG’ (Grupo de proceso de ingeniería de software) & ‘Política de Proceso’
- Ninguno es completo
- No nuevas ideas: Basadas en la experiencia
- Ninguno es aplicable a “cualquier proyecto”

Auditorías al Software: Auditorías de Proyecto y Auditorías al Grupo de Calidad. (PPT 14)

Auditorías de calidad de software: Es una actividad incluida dentro de las disciplinas de soporte, lo cual implica una evaluación **independiente** (realizada por un grupo de trabajo que no están relacionado con el equipo del proyecto) de productos o procesos de software que permiten asegurar el **cumplimiento** con estándares, lineamientos, especificaciones y procedimientos basados en un criterio objetivo incluyendo documentación. Las auditorías implican esfuerzo y costo para los proyectos, sin embargo, sus beneficios son superiores, son un instrumento para el aseguramiento de calidad en el software

Beneficios de la auditorías

- Se obtienen opiniones objetivas e independientes
- Permiten identificar áreas de insatisfacción potenciales del cliente
- Permiten asegurar que se están cumpliendo con las expectativas
- Permite dar visibilidad sobre los procesos de trabajo
- Da visibilidad a la gerencia sobre los procesos de trabajo
- Determinan que se implementen de manera efectiva: el proceso de desarrollo organizacional y del proyecto, y las actividades de soporte

Tipos de auditorías: Analizaremos 3:

1) Auditoría de proyecto: Es responsable de ver que el proyecto se haya ejecutado con el proceso que se **definió**. Esto bajo la premisa de que en un proyecto se debe realizar lo que define el proceso, **adaptado** al contexto particular de dicho proyecto. Lo que se hace es tomar evidencias de lo que se está haciendo y contrastarlo con lo documentado en la ERS, arquitectura, diseño, etc. Acá puede haber diferencias respecto a metodologías tradicionales o empíricas. En **SCRUM**, por ejemplo, es el **equipo** quien realiza las auditorías en la ceremonia de Sprint Retrospective. En cambio, en las metodologías **tradicionales**, el auditor debe ser **externo** al proyecto y a veces a la empresa. Estas auditorías se llevan a cabo de acuerdo a lo establecido en las **PACS** (plan de aseguramiento de calidad de software), el objetivo es verificar objetivamente la consistencia del producto a medida que evoluciona a lo largo del proceso de desarrollo.

2) Auditoría de configuración funcional: Valida que el producto cumpla con los **requerimientos**. La auditoría funcional compara el sw que se ha construido (incluyendo sus formas ejecutables y su documentación disponible) con los requerimientos de software especificados en la **ERS**. El propósito es asegurar que el código implementa sólo y completamente los requerimientos y las capacidades funcionales especificadas en la ERS.

3) Auditoría de configuración física: Valida que el **ítem de configuración** cumpla con la documentación técnica que lo describe (esto permite trazabilidad y satisfacción de los requerimientos). Este tipo de auditoría compara el **código con** la **documentación** de soporte, su propósito es asegurar que la documentación que se entrega es consistente y describe correctamente al código desarrollado. El PACS debe indicar a la persona responsable de realizar esta auditoría. En caso de identificarse desviaciones, el software solamente podrá entregarse cuando estás hayan sido solucionadas.

3 Roles en una auditoría

1) Auditado: Es quien participa activamente en el proceso de auditoría. Es quien **propone** la fecha para su realización, entrega las evidencias necesarias, responde las consultas del auditor y colabora en la identificación y resolución de posibles desviaciones. Además, tiene la responsabilidad de analizar el reporte de auditoría, responder formalmente y elaborar planes de acción para corregir las deficiencias señaladas. Por lo general, este rol lo desempeña el líder de proyecto, aunque puede ser asumido por otra persona

2) Auditor: Puede ser una o más personas **externas** al proyecto que se está evaluando. Generalmente, este rol lo asume el grupo de aseguramiento de calidad, encargado de brindar soporte y llevar a cabo auditorías de este tipo, entre sus responsabilidad:

- Coordinar la fecha de realización de la auditoría
- Comunicar el alcance y los objetivos de la misma al equipo auditado
- Recolectar y analizar evidencias objetivas, relevantes y suficientes para emitir conclusiones
- Llevar a cabo la auditoría, verificando el cumplimiento de los procesos y estándares establecidos
- Elaborar el reporte de auditoría, donde se detallan los hallazgos, observaciones y posibles desviaciones
- Dar seguimiento a los planes de acción acordados con el auditado, asegurando que las deficiencias identificadas sean corregidas de manera efectiva

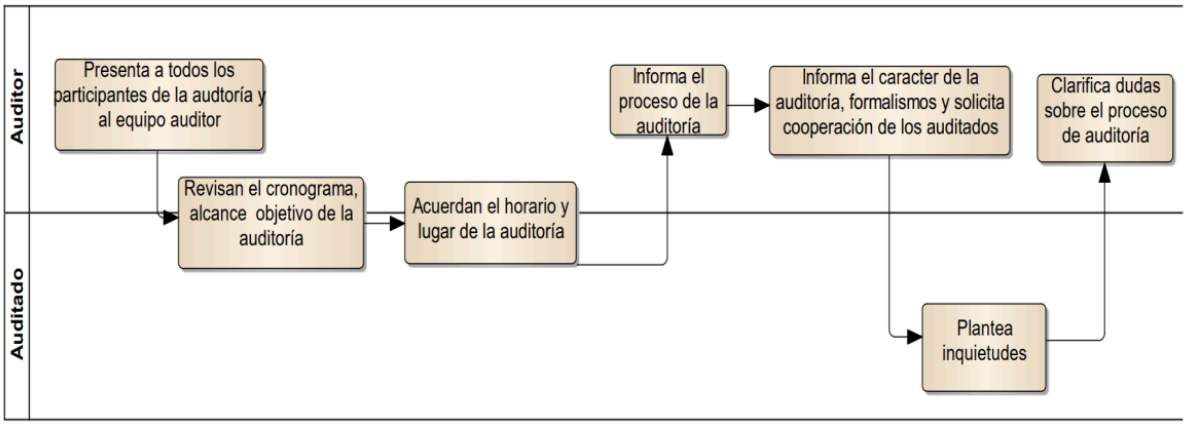
3) Gerente de SQA: Es quien prepara el plan de auditoría, calcula su costo, asigna recursos, resuelve no-conformidades entre auditor y auditado (gestión de auditoría)

Proceso y etapas de Auditorías

1) Preparación y planificación: No son sorpresa. El auditado solicita la auditoría y junto con el auditor definen la fecha, el alcance y objetivo, acordando los participantes de esta.
Se deben definir métricas de auditoría, como por ejemplo: esfuerzo por auditoría, cantidad de desviaciones, duración de la auditoría, cantidad de desviaciones clasificadas por auditor de CMMI, etc. Además el auditor solicita y revisa la documentación preliminar (opcional), y el auditado prepara la documentación e información que será requerida para la auditoría

2) Ejecución: El auditor escucha lo que la gente dice que hace y luego ve la documentación, pide evidencia (lo que debería estar haciéndose).

- **Reunión de apertura:** se informa cómo será el proceso, indicando el lugar y la hora.

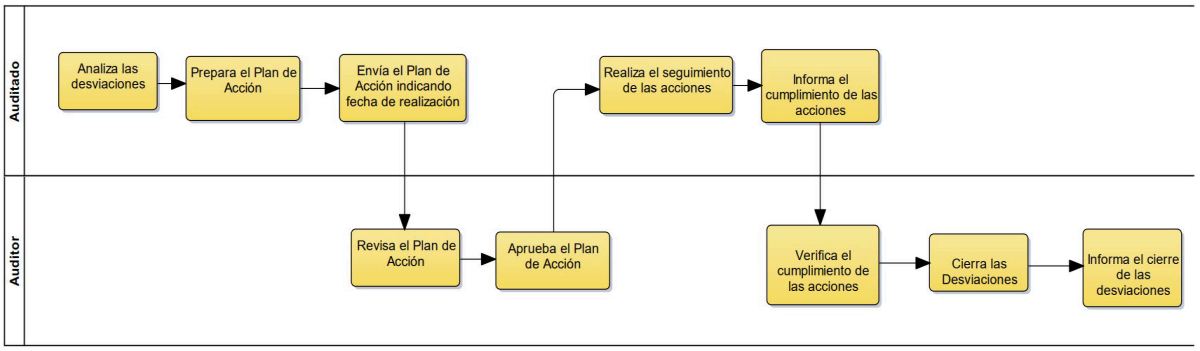


- **Ejecución:** se responden las preguntas, se muestra la evidencia y se completa el checklist. El auditor escucha lo que la gente dice que hace y luego ve la documentación (lo que debería estar haciéndose).

3) Análisis y reporte de resultados: El auditor realiza el reporte de resultados y el auditado analiza la respuesta, puede o no estar de acuerdo con algunas prácticas y se deja asentado el documento final. Se evalúan los resultados, se hace una reunión de cierre y se hace entrega del reporte final.

- **Evaluación de Resultados:** El auditor realiza el informe preliminar.
- **Reunión de cierre:** Se presenta el reporte preliminar, el auditado solicita clarificación de resultados, el auditor se los explica y se acuerdan los resultados, además de definir la fecha en la que enviará el reporte final
- **Entrega de reporte final:** El auditor actualiza el reporte preliminar con las observaciones de la reunión de cierra y envía el resultado final al auditado

4) Seguimiento: Se analizan las desviaciones y prepara un plan de acción que se envía al auditor para que lo revise y apruebe. En función del acuerdo entre auditado y auditor, puede que el auditor haga un seguimiento de las desviaciones que encontró hasta que considere que han sido resueltas



Herramientas y técnicas utilizadas en auditorías:

- **Checklists:** Tienen preguntas tipo para garantizar que independientemente de quién haga la auditoría el foco de las cosas que se controlan sea el mismo. Son de mínima, es decir, se debe garantizar de mínimo contestar estas preguntas, pero el auditor puede solicitar más cosas (hacer más preguntas).
- **Muestreo:** Consiste en seleccionar una muestra representativa de los productos y/o procesos a auditar.
- **Revisión de registros**
- **Herramientas automatizadas**

Resultados/hallazgos de una auditoría:

- **Buenas prácticas:** Cuando es algo superador a lo definido que tenemos que hacer. Mucho mejor de los esperado. Ejemplo: se armó una muy buena herramienta para gestionar el plan de proyecto para que la gente pueda accederlo y mejorarlo.
- **Desviaciones:** Cualquier cosa que no se hizo o que no se hizo cómo el proceso lo definió. Hay dos grados:
    - **No Adecuado:** Lo que realmente está mal.
    - **Necesita Mejora:** Si lo estás haciendo, pero necesita mejorarse.

  Requiere un plan de acción por parte del auditado.
- **Observaciones:** Son cosas que se advierten por el auditor que no llegan a ser desviaciones pero que pueden llegar a ser riesgosas las prácticas y pueden llegar a generar un problema, por eso se destacan a consideración del equipo para que ellos decidan si hacerlo o no. Algo para revisar. Deberían mejorarse, pero no requieren un plan de acción. Ejemplo: técnica de estimación mala.

Reporte de auditoría: En este documento se deben informar las siguientes desviaciones:

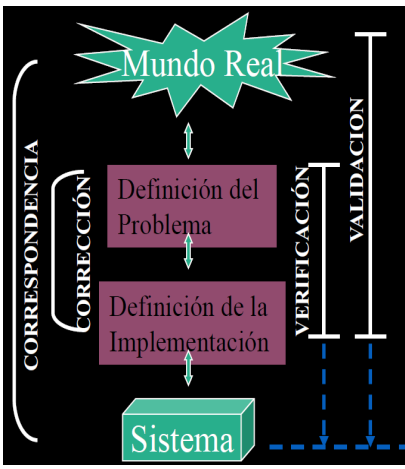
- Cualquier desviación que resulta en la disconformidad de un producto respecto de sus requerimientos
- Cualquier desviación al proceso definido o a los requerimientos documentados

Métricas de auditoría: Cada organización establece según sea apropiada o no:

- Esfuerzo por auditoría.
- Cantidad de desviaciones.
- Duración de auditoría.
- Cantidad de desviaciones clasificadas por PA de CMMI

Calidad de Producto: Planificación de pruebas para el software- Niveles y tipos de pruebas para el software. Técnicas y herramientas para probar software. (PPT 13)

Verificación y validación: Son actividades esenciales dentro del ciclo de vida del desarrollo de software. Su objetivo es asegurar que el producto se construya correctamente y que además sea el producto correcto



- **Verificación:** responde a la pregunta **¿estamos construyendo el producto correctamente?**. Es decir, se enfoca en comprobar que el software cumple con las especificaciones, normas y estándares definidos. Abarca actividades como revisión de código, inspecciones y pruebas para detectar errores técnicos o de implementación
- **Validación:** responde a la pregunta **¿estamos construyendo el producto correcto?**. Busca garantizar que el sistema desarrollado satisfaga las necesidades y expectativas del usuario o cliente. Implica revisar que los requerimientos funcionales y no funcionales sean los que el usuario realmente necesita Estas actividades se llevan a cabo durante todo el ciclo de vida del proyecto, desde la revisión de los requerimientos hasta las pruebas finales del sistema

Falla: Error en un producto de trabajo.
Producto de trabajo: Salida de cualquier actividad correspondiente al ciclo de vida de desarrollo.
¿Por qué existen las fallas?: Problemas de comunicación y limitaciones de memoria.



Principios:

- **Prevenir es mejor que curar.**
- **Evitar es más efectivo que eliminar:** Un proceso bien definido y revisiones tempranas reducen la aparición de defectos
- **La retroalimentación enseña efectivamente:** Cada revisión o prueba debe generar aprendizaje para mejorar el proceso y evitar errores futuros
- **Priorizar lo rentable:** Se deben enfocar los esfuerzos en los controles de calidad que aporten más valor al producto
- **Olvidarse de la perfección:** No se puede eliminar absolutamente todos los errores, lo importante es que el producto cumpla con los objetivos
- **Enseñar a pescar, en lugar de dar el pescado.**

Existen dos aproximaciones complementarias: Revisiones técnicas - Pruebas de SW.
Revisiones técnicas - Peer review: Son actividades realizadas por colegas del **mismo** equipo con el propósito de mejorar la calidad del software mediante la detección **temprana** de errores en cualquier artefacto generado durante el desarrollo, ya sea código, requerimientos, diseño, arquitectura, análisis de riesgos, estimaciones o planes. Se trata de un proceso **estático de verificación y validación**, lo que significa que no corrige errores, sino que busca **identificarlos** antes de que avancen a etapas posteriores, evitando así retrabajo y costos innecesarios. El objetivo principal de estas revisiones es introducir el concepto de verificación y validación dentro del equipo, promoviendo una cultura de mejora continua y de compromiso con la calidad. Además, motiva a los integrantes a realizar un trabajo más cuidadoso y profesional, sabiendo que será revisado por sus pares.
Un aspecto fundamental es que nunca se evalúa a la persona que creó el artefacto, sino al **artefacto** en sí. Esto fomenta un ambiente de colaboración y aprendizaje, donde los errores se entienden como **oportunidades de mejora** y no como motivo de culpa. La clave está en generar una cultura de apoyo mutuo y confianza dentro del equipo. Esta práctica trasciende cualquier metodología o filosofía de desarrollo y es ampliamente adoptada incluso por metodologías ágiles, que la utilizan para reemplazar auditorías formales por revisiones entre pares, evitando así la necesidad de involucrar auditores externos

Ventajas:

- Pueden descubrirse muchos errores
- Pueden inspeccionarse versiones incompletas
- Pueden considerarse otro atributos de calidad

Desventajas:

- Es difícil introducir inspecciones formales
- Sobrecargan al inicio los costos y conducen a un ahorro sólo después de que los equipos adquieran experiencia en su uso
- Requieren tiempo para organizarse y parece como si “ralentizará” el proceso

Tipos de revisiones técnicas

- **Formales:** Tienen un proceso definido con roles (inspecciones)
- **Informales:** Cuando no existe un proceso definido de cómo realizarlo (recorrido)

Inspecciones (formales): Estructurada de revisión del sw, en la que participan distintos roles definidos y se siguen pasos específicos para garantizar la calidad de producto. A diferencia de las revisiones informales, las inspecciones requieren planificación, registro de resultados y documentación del proceso. Se utiliza un checklist que sirve como guía para no olvidar ningún aspecto importante al momento de evaluar el artefacto (por ejemplo un documento de requerimientos, un diseño o porción de código). Este **checklist** ayuda a asegurar que todos los elementos se revisen de manera sistemática y consistente. Además, se recogen **métricas**, para evaluar la efectividad del proceso y aplicar mejoras en futuras inspecciones. Al finalizar, se elabora un **reporte** detallado con los hallazgos y conclusiones, el cual sirve como base para analizar los defectos encontrados y definir acciones correctivas. Tiene los siguientes **objetivos**:

- Detectar errores en etapas tempranas del ciclo de vida del software
- Verificar que el software cumple con los requerimientos establecidos
- Garantizar que el desarrollo se ajusta a los estándares definidos por la organización o el proyecto
- Asegurar uniformidad en la forma en que se desarrolla el software dentro del equipo
- Facilita la gestión del proyecto, ya que al tener procesos más controlados y menos defectos acumulados, los proyectos se vuelven más predecibles y manejables

Estas actividades se consideran time-boxing, lo que significa que tienen un **tiempo límite** claramente establecido, para evitar que se extiendan demasiado y afecten la productividad

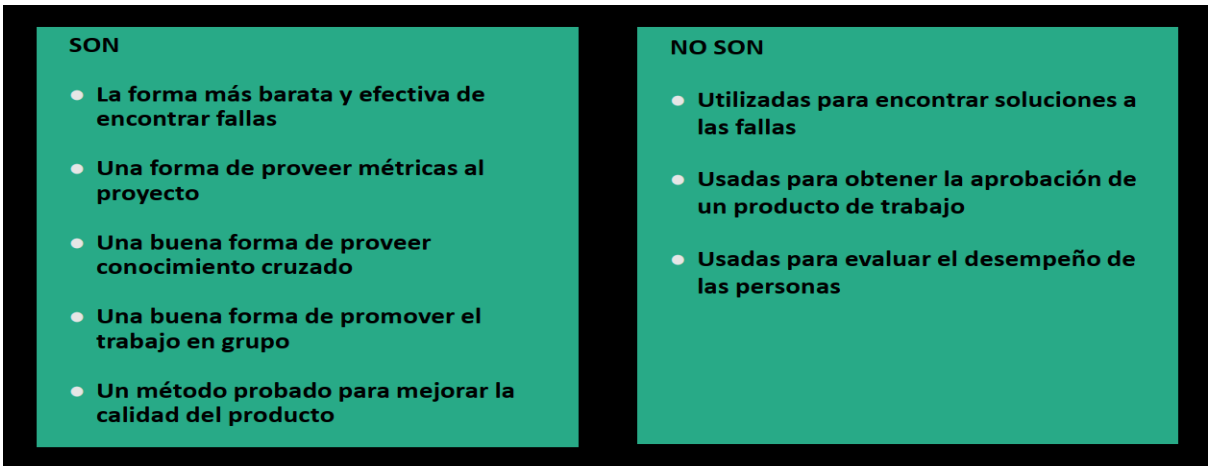
Walkthrough (recorrido-informal): Es una técnica de análisis estático utilizada para revisar un producto de sw (como código, diagramas, documentación o diseño) **antes** de que sea probado o ejecutado. Su principal característica es que no sigue un proceso formal ni estructurado, sino que se realiza de manera colaborativa e informal dentro del equipo de desarrollo. En esta técnica, el auto del artefactos (diseñador o programador) presenta su trabajo al resto del equipo, explicando cómo funciona y cuáles fueron las decisiones tomadas. Los participantes (otros desarrolladores, testers, analistas u stakeholder) hacen preguntas, sugieren mejoras y señalan posibles errores o incumplimientos de estándares. A diferencia de las inspecciones, no requiere de un checklist, ni documentación formal, ni recolección de métricas. Su objetivo es más formativo y práctico, enfocado en mejorar el producto y fortalecer el conocimiento del equipo sin generar sobrecarga. **Objetivo**:

- Mínima sobrecarga y rápido retorno
- Capacitación de desarrolladores

En los enfoques **ágiles**, esta práctica es muy común porque se adapta perfectamente a su filosofía: colaboración constante, comunicación abierta y mejora continua. En lugar de realizar auditorías formales o revisiones externas, los equipos suelen hacer revisiones rápidas entre colegas al finalizar cada sprint, donde se analizan los resultados obtenidos

5 Roles participantes: Al ser una técnica **formal**, debe contar con los siguientes roles:
1) Autor: Creador o encargado de mantener el producto a inspeccionar. Inicia el proceso seleccionando a un moderador y junto a este eligen al resto de los roles. Entrega el producto a ser inspeccionado al moderador
2) Moderador: Planifica y lidera la revisión. Trabaja junto al autor para elegir los demás roles. Entrega el producto a inspeccionar al inspector 2 días antes de la reunión. Coordina la reunión sin conductas inapropiadas y realiza un seguimiento sus defectos encontrados
3) Anotador: Registra los hallazgos de la inspección y realiza el reporte de la revisión
4) Lector: Lee el producto a ser inspeccionado. Evita que se disperse el resto
5) Inspector: Examina el producto antes de la reunión para encontrar defectos. Registra sus tiempos de preparación, todos pueden ser inspectores, todos pueden inspeccionar

Las revisiones técnicas:



Etapas del proceso de inspección

1. **Planificación:** El moderador, a pedido del autor, planifica la inspección definiendo el lugar, duración (menor a 2 horas ya que requiere esfuerzo intelectual) y los roles.
2. **Visión general:** El autor realiza una descripción del producto a inspeccionar. (OPC)
3. **Preparación:** Es la preparación de cada rol para la reunión. Cada rol adquiere una copia del producto de trabajo que deberá leer y analizar, con el fin de encontrar potenciales defectos. Esta preparación hace productiva la reunión de inspección
4. **Reunión de inspección:** El equipo realiza un análisis para recolectar los potenciales defectos previos y descartar falsos positivos. La reunión finaliza con una conclusión acerca de si se acepta o no el producto de trabajo inspeccionado. Más informe detallando que se revisó, por quien, que se descubrió y que se concluyó.
5. **Corrección:** El autor realiza las correcciones de los defectos encontradas.
6. **Seguimiento:** Dependiendo de la gravedad puede existir un proceso de re-inspección. En caso de que los defectos sean graves, se realiza nuevamente una inspección. De lo contrario, si el defecto era muy simple, el autor simplemente lo corrige. Otros defectos que no implican una re-inspección, pueden implicar que el autor se reúna con el moderador únicamente para tratar las correcciones realizadas.

Definición de estándares: Un aspecto importante del aseguramiento de calidad es la definición o selección de estándares que deben aplicarse al proceso de desarrollo de software o al producto de software. Los estándares son importantes por **tres razones**:

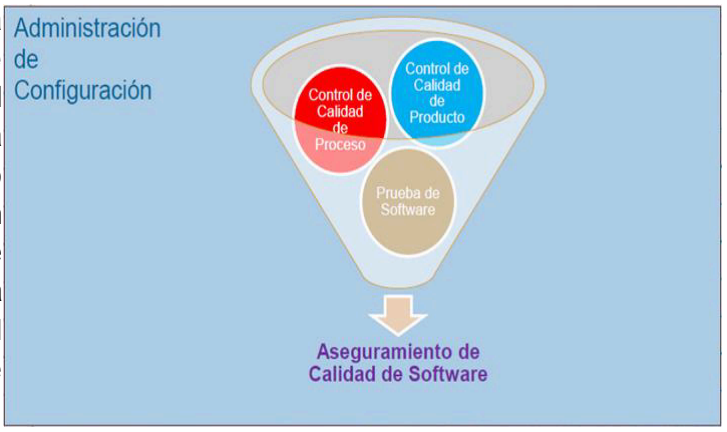
- Se basan en conocimiento sobre la mejor o más adecuada práctica para la organización. Con frecuencia, este conocimiento se adquiere sólo después de una gran cantidad de pruebas y errores. Configurarla dentro de un estándar, ayuda a la empresa a reutilizar esta experiencia y a evitar errores del pasado.
- Al usar estándares se establece una base para decidir si se logró un nivel de calidad requerido. Esto depende del establecimiento de estándares que reflejen las expectativas del usuario para la confiabilidad, la usabilidad y el rendimiento del sw.
- Los estándares aseguran que todos los trabajadores dentro de una organización adopten las mismas prácticas. En consecuencia, se reduce el esfuerzo de aprendizaje requerido al iniciar un nuevo proyecto o trabajo.

Para la gestión de calidad de software se utilizan dos estándares:

● **Estándares de producto:** Incluyen estándares de documentos y de codificación.

● **Estándares de proceso:** Establecen los procesos que deben seguirse durante el desarrollo, ej, incluyen las definiciones de requerimientos, procesos de diseño y validación.

Testing en ambientes Ágiles (PPT 09)



El **testing** de software, es una de las tareas a realizar en el ámbito del **aseguramiento de calidad de software**, en **conjunto con el control de calidad del proceso y del producto**. La calidad del software se obtiene y se logra a lo largo de todo el desarrollo de este (detectar errores en etapas tempranas es siempre menos costoso, que detectarlos después), por lo que una buena administración de configuración (SCM) es el puntapié inicial para permitir la realización de tareas de aseguramiento de calidad.

Hay que recordar que SCM permite determinar la configuración de ítems, trazabilidad, control de cambios, auditorías del producto y reportes de estado. **QA** agrega a esto, el control de calidad del **proceso**. Existen diversos estándares (ISO, CMMI, etc.) que permiten acreditar la calidad del proceso, debido a que la calidad del producto no es algo estandarizable por la variación de requerimientos de un caso a otro.

Dentro del aseguramiento de la calidad del software, existen actividades destinadas al control de calidad del proceso y del producto. Se realiza un control de calidad sobre el proceso de desarrollo, bajo el argumento de que el proceso de desarrollo posee un gran impacto en la calidad del producto. Es decir, en teoría la calidad del producto depende de la calidad del proceso que se está utilizando.

Luego de desarrollar el software, las actividades de testing revelan la calidad del Software en sí, es decir, si el producto satisface con los requerimientos definidos por el cliente en etapas anteriores. Cabe destacar que la actividad de testing no asegura la calidad por sí solo, sino que debe estar acompañada de las demás actividades. QA != Testing

No podes asegurar ni verificar la calidad, ni garantizar que no tenga errores. Se asume que el sw va a tener defectos, y lo q hace el testing es detectarlos.

La diferencia con el **tradicional**, es que en el **agilismo** tenes que detectar errores en porciones más pequeñas, en el tradicional (etapas del pud) es una etapa muy tarde en la que tenes todo el producto junto, lo que lo vuelve más caro y difícil de realizar. Las pruebas unitarias si las hace el desarrollador pero el testing debería ser distinto al que lo programa.

Definición de Testing: Proceso mediante el cual se somete a un sw o un componente de sw a condiciones específicas con el fin de determinar y demostrar si el mismo es válido o no en función de los requerimientos especificados. El testing es una actividad **destructiva** cuyo objetivo es encontrar defectos, cuya presencia es asumida de antemano en el código.
Testing de software es un proceso, o una serie de procesos, diseñados para asegurar que el código hace lo que fue diseñado para hacer, que no haga nada que no esté intencionado. Se dice que es el proceso de ejecutar un programa con la intención de encontrar fallas. El software debería ser predecible y consistente, sin presentarle sorpresas a los usuarios. El Testing es parte del QA (Quality Assurance) y forma parte del control de calidad. Se hace cuando el producto ya está construido (o lo que se desea testear), en cambio el QA se aplica en todo el ciclo de vida (las buenas prácticas en la implementación son parte de QA).
Las actividades de testing se consideran exitosas si se encuentran defectos en la ejecución de los casos de prueba. Por lo que, un software con alta calidad lograda a lo largo de todo el desarrollo (implementación de QA), dificulta encontrar defectos y puede dirigir a un testing no exitoso. Por otro lado, si el software tiene un bajo nivel de calidad, el costo de retrabajo es alto, ya que van a existir muchos idas y vueltas entre las actividades de testing y las de desarrollo, para la corrección de errores. El testing es una actividad costosa, por lo que es necesario lograr un nivel de cobertura óptima teniendo en cuenta el costo-beneficio. Nunca se podrá cubrir el 100% de las pruebas, por cuestiones lógicas de tiempo y costos. Esta cobertura se comienza a definir desde el momento en los cuales se establecen los criterios de aceptación. En caja blanca miras el codigo, es estático. En caja negra tenes que ejecutar, entonces es dinámico. Un proceso completo debería tener las dos

Principios del testing

- Es una actividad destructiva que encuentra defectos cuya presencia se asume
- Se testea con actitud negativa tratando de demostrar que algo es incorrecto
- El testing exitoso es aquel que encuentra defectos
- Siempre es necesario. NO agrega calidad.
- El costo del testing está entre un 30% y 50% del valor del producto
- Puede empezar antes de la codificación porque necesita de los requerimientos únicamente para armar los casos de prueba
- Pone en evidencia defectos, pero no agrega calidad ni garantiza que el producto no tenga errores

Principio	Explicación
1. Una parte necesaria de un caso de prueba es definir el resultado esperado.	Si el resultado esperado de un caso de prueba no ha sido predefinido, lo más probable es que un resultado erróneo se interpretará como un resultado correcto, debido a el fenómeno de "el ojo viendo lo que quiere ver", a pesar de la definición destructiva adecuada de prueba, hay

	todavía un deseo subconsciente de ver el resultado correcto.
2. Un programador debe evitar testear su propio programa.	Los propios desarrolladores “no quieren” encontrar sus propios defectos, por lo que el carácter de pruebas destructivas se deja de lado. Además, puede que el programa contenga errores por malentendidos del programador sobre el dominio, y si él mismo testea, no se van a detectar estos defectos.
3. Una empresa de desarrollo no debe testear sus propios programas.	Misma explicación que el principio 2 orientado a empresas, agregando que si las mismas empresas testean sus programas, es posible que destinen menos recursos y eviten encontrar defectos para cumplir con el calendario y los costos establecidos.
4. Cualquier proceso de prueba debe incluir una inspección minuciosa de los resultados de cada prueba.	Se deben realizar inspecciones minuciosas para detectar la totalidad de los defectos encontrados tras la ejecución de una prueba, ya que pueden surgir más de un defecto (y esto es lo que se busca) por cada caso de prueba.
5. Los casos de prueba deben escribirse para condiciones de entrada que no son válidas e inesperadas, así como para las que son válidas y esperadas.	Muchos errores que se descubren repentinamente en el desarrollo de software aparecen cuando se usa de alguna manera nueva o inesperada, y no responde cómo debería (catcheando el error)
6. Examinar un programa para ver si no hace lo que se supone que debe hacer es sólo la mitad de la batalla; la otra mitad es ver si el programa hace lo que no se supone que debe hacer	Los programas deben ser examinados para detectar defectos secundarios no deseados.
7. Evite los casos de prueba desechables, a menos que el programa sea realmente un programa de descarte.	Los casos de pruebas utilizados en el testing deben ser reproducibles, es decir, no se deben realizar casos de prueba sobre la marcha (ad-hoc) ya que es imposible reportar un defecto sin tener las condiciones y los pasos en los cuáles el defecto surgió. Además, realizar testing es muy costoso, por lo cuales los casos de prueba deben ser reutilizados para volver a testear escenarios luego de la corrección de errores.
8. No planee un esfuerzo de prueba bajo la suposición tácita de que no se encontrarán errores.	Es un error pensar de esta manera al momento de realizar software. Se debe tener en claro la definición de testing, en la cual se define que el objetivo de esta

	actividad es encontrar errores, y se presume de antemano su existencia.
9. La probabilidad de que existan más errores en una parte de un programa es proporcional al número de errores ya encontrados en esa parte.	El concepto es útil porque nos da una idea de en qué sección del programa hacer foco o asignar más recursos, si una sección particular de un programa parece ser mucho más propenso a errores que otras secciones, es recomendable realizar pruebas adicionales y es probable que encontremos más errores.
10. Las pruebas son extremadamente creativas e intelectualmente desafiantes.	Aunque existen métodos y estrategias para abarcar un mejor nivel de cobertura testing en los casos de pruebas, siempre es necesario un poco de creatividad del diseñador de estos.

Mitos del Testing

- “El testing es el proceso para demostrar que los errores no están presentes”.
- “El propósito del testing es demostrar que un programa realiza sus funciones previstas de forma correcta”
- “El testing es el proceso que demuestra que un programa hace lo que se supone que debe hacer’’

Estas afirmaciones sobre el testing son incorrectas, ya que el objetivo de testear un programa es agregar valor al producto revelando su calidad y brindando confianza en el sw, de forma más concreta, encontrar y remover defectos en el código de este. No se prueba un sistema para mostrar que funciona, sino que se comienza con la suposición de que el sw contiene defectos, y se realiza el testing para encontrar la mayor cantidad de ellos posible.
Por otro lado, un programa puede hacer lo que se supone que debe hacer, y aun así, contener defectos. Es decir, un error está claramente presente si un programa no hace lo que se supone que debe hacer, pero los errores también están presentes si un programa hace lo que no se supone que debe hacer.

¿Cuánto Testing es suficiente?: El testing exhaustivo es imposible por la cantidad de tiempo que requiere. Cuánto testing depende del nivel de riesgo o costo asociado. Los riesgos permiten definir prioridades de que se debe testear primero y con qué esfuerzo. El criterio de aceptación se utiliza para decidir si una determinada fase de testing se completó.

- Costos.
- % de tests corridos sin fallas.
- Inexistencia de defectos de una determinada severidad.
- Pasa exitosamente el conjunto de pruebas diseñado y la cobertura estructural.
- Good Enough: Cierta cantidad de fallas no críticas es aceptable.
- Defectos detectados es similar a la cantidad de defectos estimados.

El criterio de aceptación sirve para definir y negociar en ágil con el Product Owner y en tradicional con el Líder del Proyecto a cuántos defectos son aceptables para terminar.

Conceptos importantes

Defecto vs error: La diferencia entre ambos conceptos es el **momento** en el cual se detectan y solucionan los errores o defectos. Un error es detectado y corregido en una misma etapa, y un defecto es un error que se traslada de una etapa a otra etapa posterior en la cual se introdujo. El testing encuentra defectos, ya que son errores que surgieron en etapas anteriores, pero se detectan en la etapa de prueba. Hay que aclarar que ambos conceptos pueden o no generar fallas o mal funcionamiento en el sistema, (ej colores)
En otras palabras: Fallo es un concepto distinto al de error vs defecto. Falla es un mal funcionamiento del sistema. Es cuando llega al software. Ocurre como consecuencia de que se introdujo un defecto. No todos los defectos producen fallas en el software. El error y defecto son nuestros, provocan fallas en el sistema.
Defecto: Posee 2 características, que permiten catalogarlos en la etapa de testing:

- **Severidad:** Define la gravedad del defecto, y es determinada por la persona que realiza el testing, por lo que esta característica es de carácter técnico. El valor de la severidad se asigna dependiendo de la siguiente escala
    - Bloqueante → el defecto no permite continuar con la ejecución del sistema.
    - Crítico → el sistema funciona, pero la funcionalidad que se está testeando tiene un defecto crítico. Encima no tenes otra forma de hacerlo.
    - Mayor → la funcionalidad que se está testeando funciona, pero no de forma correcta. Tenes otra forma de hacerlo.
    - Menor → la funcionalidad se ejecuta correctamente, pero con advertencias erróneas o errores de baja importancia. No impide el funcionamiento
    - Cosmética → formato de fechas, formato de números, distribución de componentes en una GUI, temas de presentación de interfaces, etc.
- **Prioridad:** Define el impacto del defecto en la funcionalidad para el negocio, y permite ordenar la atención de los defectos según las necesidades del cliente. La asigna el usuario final cuando la detecta, pero esto no debería pasar. La realidad es que esto si pasa, el usuario final detecta defectos y dice su prioridad. Una escala posible para la prioridad puede ser:
    - Urgencia - Alta- Media - Baja
- Cada equipo los ajusta a estos tipos, la profe recomienda no hacer escalas impares, ya que hay una tendencia de ponerlo al medio.

Estas características son independientes entre sí, ya que varían dependiendo del tipo de negocio y de sistema que se está desarrollando. Ej, un defecto cosmético puede tener baja prioridad para un sistema de inventarios, pero puede tener una alta prioridad si se trata de un sistema de una empresa de marketing, en donde la presentación es algo importante.

Casos de prueba: Son una secuencia de pasos que hay que seguir para obtener un resultado esperado, y se tienen que dar ciertas condiciones previas que fijan una situación para que se pueda ejecutar la prueba. Es el artefacto más importante del Testing. Este se hace una vez, pero sirve para realizar infinitas pruebas, debiendo obtener en todas el mismo resultado esperado. Se trata de minimizar la cantidad de casos de prueba maximizando la cantidad de defectos encontrados. El objetivo de los casos de prueba es descubrir defectos.
Los casos de prueba salen de los requerimientos, permitiendo hacer tanto la verificación como la validación. Está compuesto por: un **objetivo** (lo que se desea controlar), **condiciones de prueba** (Datos de entrada y de entorno que deben estar presente para llevarse a cabo la prueba) **y un resultado esperado.**

Ciclos de prueba: Es la ejecución de un conjunto de casos de prueba en una versión determinada del producto. Se tienen 2 ciclos, y el primero es conocido como ciclo 0. El ciclo 0 siempre es manual, es donde se configura todo y a partir del ciclo 1 ya se pueden automatizar las pruebas. En caso de detectarse defectos, el producto vuelve a desarrollo para su corrección. Si existe una cantidad alta de ciclos de prueba, es probable que QA no sea bueno, por lo que deberían revisarse ciertos aspectos acerca de la calidad del proceso y producto, de manera de evitar tener grandes cantidades de ciclos, que se traducen en mucho retrabajo, lo cual a su vez conduce a incrementar los costos de desarrollo.

Ciclo de pruebas con regresión: Plantea la necesidad de ejecutar exhaustivamente todos los casos de prueba en cada ciclo de prueba, como si fuesen el ciclo 0. Este planteo es el ideal, pero el menos utilizado, por cuestiones de practicidad y tiempo. El otro enfoque, consiste en aplicar una prueba exhaustiva de todos los casos de prueba en el ciclo 0, pero a partir del ciclo 1 (si el incremento o producto vuelve nuevamente a testing) ejecutar sólo los casos de prueba asociados a defectos encontrados previamente, y no los casos de prueba que hayan pasado sin encontrar defectos. Esto permite agilizar el proceso de pruebas, pero estadísticamente es muy probable que al arreglar un defecto reportado por testing, en el desarrollo de su corrección se introduzcan nuevos errores en otras partes del producto. Por lo que, si no se prueban todos los casos, a pesar de que previamente no se hayan detectado defectos, pueden encontrarse nuevos, debido al proceso descrito anteriormente.
Una solución a esta encrucijada es no aplicar regresión (ejecutar sólo los casos de prueba con defectos asociados), y cuando finalmente estén todos los defectos “corregidos”, hacer una ejecución de todos los casos de prueba (como si se aplicara regresión), para aumentar la confianza de que no se han introducido nuevos errores en las correcciones. La automatización del testing permite hacer tantos ciclos de prueba con regresión como se deseen. La **desventaja** es el proceso de automatizar la ejecución de los casos de prueba, pero se puede ver ese esfuerzo de automatización como el esfuerzo propio del ciclo 0 manual, y luego esa automatización permite ejecutar los casos de prueba N veces. En empresas donde el core de negocio no es hacer sw, conviene asumir ese esfuerzo y automatizar las pruebas. La mejor estrategia es aplicar regresión, porque no existe un punto medio al ser un método de caja negra, ya que no se puede saber con certeza lo que va a afectar al corregir un defecto y en dónde impactará esa corrección. Pero en la práctica se utiliza sin regresión, debido a los costos y tiempos que implica ese proceso. La estadística dice que cada 1 defecto se generan 3 más, por eso el ciclo 1 se tiene que probar de la misma forma que el ciclo 0, por eso se tiene que usar regresión, la automatización intenta hacer eso, es muy importante con la automatización agregarle regresión

Niveles de prueba

Determinan el foco o la granularidad de la prueba que se está por ejecutar. En general, primero se comienzan probando componentes pequeños, y luego se integran en pruebas de mayor granularidad. Esto es al revés de cómo se desarrollan los componentes

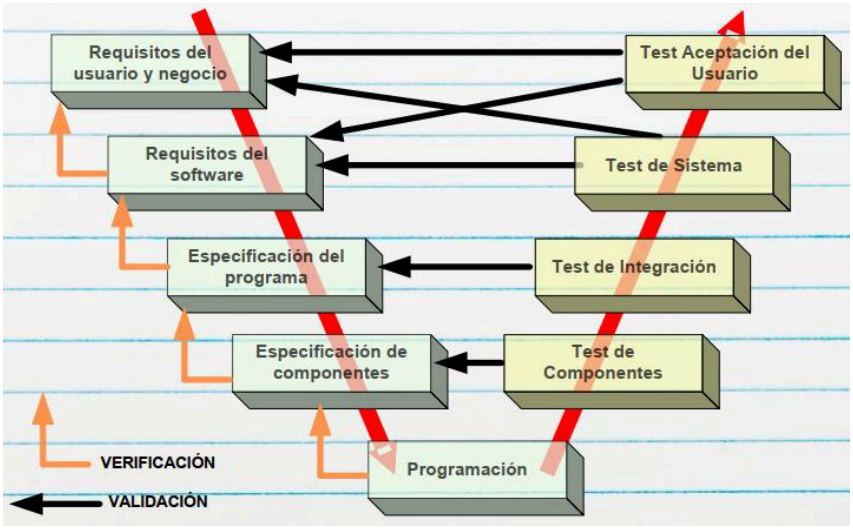
Pruebas unitarias: Las hace el desarrollador y están incluidas en el DoD. Son pruebas que se realizan sobre un componente, de manera independiente a los otros. Se hacen teniendo acceso al código fuente, y se pueden usar herramientas para realizarlas (automatización, depuración). Se suelen reparar los errores apenas se encuentran, sin registrarlos.

Pruebas de integración: El propósito de la ejecución de estas pruebas es verificar el funcionamiento de dos o más componentes juntos que se relacionen entre ellos, es decir, clases en las cuales existe una interacción a modo de peticiones, a través de sus interfaces (boundaries). El resultado de estas pruebas es el build (el **CI** o continuous integration), el cual luego se envía al equipo de testing y es lo que se prueba en siguientes etapas. Se suele llevar a cabo una prueba de integración incremental, desde lo más general (que abarca más componentes) a lo más específico (top-down) o al revés (bottom-up).

Pruebas de sistema o de versión: En estas pruebas se realizan testeos sobre la versión de un incremento de producto o de un producto (dependiendo si se usa Agile o métodos tradicionales), y deben realizarlas personas ajenas al desarrollo de los programas. Se llevan adelante siguiendo un proceso sistemático y metodológico, que permite encontrar defectos que puedan ser reproducibles y reportar de qué manera ocurre el defecto detectado, es decir, cual es el camino de pasos que hay que seguir para encontrar el error. Basadas en casos de prueba. Se trata de emular de la mejor manera posible un entorno de trabajo idéntico al entrono real en el que se usara el SW. Se llevan a cabo pruebas del funcionamiento real y cotidiano del producto. Abarca requerimientos funcionales y RNF.

Pruebas de aceptación de usuario: Se realizan en el despliegue y debería realizarlas el usuario para verificar que se cumple con lo que el mismo requirió. Generando comodidad y confianza sobre el mismo (su objetivo principal no es encontrar fallas). También busca reproducir un entorno de producción. Comprende tanto la prueba realizada por el usuario en ambiente de laboratorio (pruebas alfa), como la prueba en ambientes de trabajo reales (pruebas beta). En la teoría, los usuarios arman sus pruebas de usuario. En la práctica, las arma Testing. En la teoría, además del usuario, deben estar presentes el gerente de testing, el líder del proyecto y empleados con roles funcionales. En Agile, además del usuario, deben estar todos los miembros del equipo (Sprint Review)

Modelo en V: Es un enfoque utilizado para determinar cuándo se está en condiciones de realizar determinadas actividades dentro del proceso de desarrollo de software, según la etapa en la que se encuentra el proyecto. En el lado izquierdo de la V se representan las etapas de desarrollo y verificación, y el lado derecho las etapas de pruebas y validación. En la base se encuentra la programación, actúa como punto de conexión entre ambas partes.
Izquierda: Se identifican los req del usuario y negocio, se traducen en requisitos del sw. Se realiza la especificación del programa, que incluye el diseño y arquitectura, y finalmente la especificación de componentes, donde se detallan las funciones individuales del sistema. Cada una de estas fases tiene una prueba correspondiente en el lado derecho del modelo
Derecha: Se encuentran las pruebas que validan el trabajo realizado en cada etapa de desarrollo. El proceso comienza con el testing de componentes o pruebas unitarias, donde se verifica el correcto funcionamiento de cada módulo (o unidad de código). Luego se pasa al test de integración, que evalúa la interacción entre los distintos módulos del sistema. Se realiza el test de sistema, en el que se prueba el sw completo para confirmar que cumple con los requisitos técnicos establecidos. Se lleva a cabo el test de aceptación del usuario, donde se valida que el producto final satisfaga las necesidades y expectativas del cliente. Las pruebas se realizan en un orden de granularidad inverso al del desarrollo, mientras el desarrollo avanza desde lo más general hacia lo más específico, las pruebas se ejecutan en sentido contrario: comienzan por los componentes más pequeños y detallados y concluyen validando el sistema completo frente al usuario



4 Ambientes / entornos de testing y su proceso

Son todos los recursos, tanto hardware como software, que se requieren para poder trabajar con el producto. Existen distintos ambientes en el desarrollo de software, los cuales poseen distintas necesidades, según la etapa de desarrollo en la que se encuentra el software. Los que desarrollan y los que prueban no pueden trabajar en el mismo entorno.
Desarrollo: Implica hardware y software necesarios (librerías, extensiones, IDEs, compiladores, etc) para poder desarrollar y desplegar el producto para luego utilizarlo. En este ambiente se realizan las pruebas unitarias (y también las de integración por lo general)

Prueba: Es el ambiente que utilizan los testers para llevar a cabo las pruebas, y los desarrolladores no deben tener acceso. Se realizan normalmente las pruebas de sistema

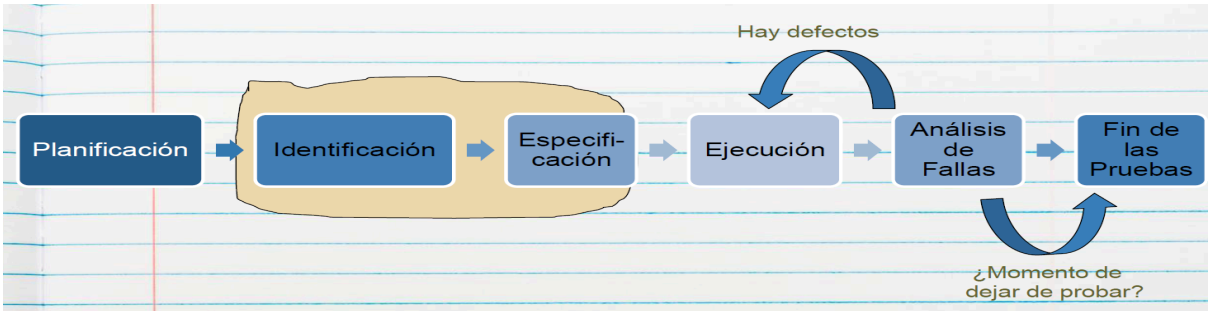
Preproducción: Es un ambiente que busca replicar las mismas condiciones que el ambiente productivo, con el objetivo de comprobar que el producto funcionará correctamente una vez que sea desplegado en producción. Acá se llevan a cabo las pruebas de aceptación, donde se valida el comportamiento del sistema en condiciones lo más similares posibles a las reales. Actúa como una instancia intermedia entre el desarrollo y la producción, permitiendo detectar posibles fallos o diferencias de configuración antes de que el software sea liberado para su uso final

Producción: Es el entorno real en el que el software se encuentra en funcionamiento y es utilizado por los usuarios finales para llevar a cabo sus tareas o actividades. Es aquella versión que ya fue validada, aprobada y puesta en marcha para su uso cotidiano. En este entorno se ejecuta la versión final del producto, que ya superó todas las etapas previas (desarrollo, pruebas y validaciones). Por esta razón, no se realizan pruebas en producción, ya que cualquier error o fallo podría tener consecuencias graves. El software que llega a producción debe cumplir con los criterios establecidos en la DoD, es decir, debe estar completamente terminado, probado, documentado y listo para su uso sin generar riesgos

Proceso de pruebas: El Testing es un proceso definido (que se lleva a cabo durante todo el ciclo de vida del producto), por lo que está compuesto de etapas detalladas.

Testing Ad Hoc: se realiza testing sin ningún proceso definido, perdiendo la trazabilidad ya que no se registra el paso a paso de lo que se probó, sino que se prueba libremente sin llevar ningún registro.
Generalmente el proceso de pruebas es estándar (puede tener algunas variaciones, pero sigue una estructura general).
Un ejemplo es: Planificacion de pruebas (saca el artefacto de plan de pruebas) -> diseño de CP (casos de prueba) -> Ejecución (lo que le entra es la versión y los diseño de cp) -> reporte defectos (en ciclo con ejecución) y despues finaliza
Hay que separar las primeras dos etapas. Las primeras dos no necesitas tener el codigo para hacerlo entonces podes adelantar trabajo, ya con la ERS.





En un ciclo cero se automatiza todo, lo ideal sería que tenga dos ciclos, partiendo de la base del concepto, lo ideal sería:

- ciclo cero encuentro defectos (por definición, mínimo 1)
- ciclo uno no encuentro defectos

1) Planificación: Esta etapa define cómo se integrará el testing dentro del plan general del proyecto y establece la elaboración del plan de pruebas. En esta fase se determinan los recursos necesarios, los riesgos asociados, los criterios de aceptación, los entornos que se deberán emular, las responsabilidades de cada integrante del equipo de pruebas y el cronograma de ejecución. El resultado de esta etapa es un documento formal llamado plan de pruebas, que debe incluir: Riesgos y objetivos del testing - Estrategia de testing - Recursos -Criterio de aceptación

2) Diseño (identificación y especificación de casos de prueba): A partir de las bases establecidas durante la planificación del testing, se identifican los datos necesarios y se diseñan y priorizan los casos de prueba que se ejecutarán. Aquí se determina qué entornos se utilizarán, cómo se llevarán a cabo las pruebas, quién será responsable de realizarlas y en qué momento se ejecutarán. Además se analiza si los requerimientos son testeables, es decir, si pueden ser verificados mediante pruebas concretas. Finalmente, se define si se aplicará o no regresión, dependiendo de la naturaleza del proyecto y del alcance de las pruebas planificadas.

3) Ejecución: En esta fase se llevan a cabo los casos de prueba, registrando los resultados obtenidos y comparándolos con los resultados esperados. A partir de esta comparación se genera un reporte de defectos, en el que se detallan los errores encontrados junto con las condiciones y entornos en los que se produjeron. El objetivo es automatizar la mayor cantidad posible de tareas dentro de la ejecución, con el fin de reducir tiempos y costos. Esta etapa incluye la creación de los datos necesarios para las pruebas, la configuración y verificación del ambiente de prueba, la automatización de procesos, la ejecución de los casos de prueba y el registro y análisis de los resultados, comparando los valores reales con los esperados para evaluar la calidad del producto

4) Análisis de fallas: En esta etapa se realiza el seguimiento de los defectos detectados durante las pruebas, verificando que cada uno haya sido corregido correctamente hasta el cierre total de los casos de prueba. Además, se evalúa el cumplimiento de los criterios de aceptación, se verifican los entregables y se confirma que las correcciones no hayan generado nuevos errores. También se elaboran reportes finales de prueba, que son presentados a los interesados del proyecto, detallando los resultados obtenidos, el estado de los defectos y el nivel de calidad alcanzado. Finalmente, se lleva a cabo una evaluación general de las actividades de testing, identificando lecciones aprendidas y oportunidades de mejora para futuros proyectos

5) Fin de las pruebas: En las organizaciones con mayor madurez en sus procesos de calidad, las pruebas finalizan únicamente cuando ya no existen defectos bloqueantes, críticos o mayores y los defectos menores o cosméticos son mínimos y no afectan el funcionamiento del producto. Estos últimos suelen detallarse en la nota de versión. Al concluir esta etapa, se puede elaborar un informe final de pruebas (opcional), en el que se resumen los resultados obtenidos, el estado del producto y las observaciones relevantes antes de su liberación

Artefactos de testing

Plan de pruebas: Este plan, es análogo al plan de proyecto que se elabora en el enfoque tradicional (es decir, no forma parte de ese plan). Acá se definen:

- Datos necesarios para las pruebas
- Recursos destinados a las pruebas
- Herramientas a utilizar
- Se se aplicará regresión o no (o en el diseño)

Para la confección de este plan no es necesario el código, sino que sólo se necesitan los requerimientos, en el formato que sea que se encuentren (ERS o casos de uso en los métodos tradicionales, user stories en ágil)

Casos de prueba: Los casos de prueba son el elemento fundamental del proceso de testing, ya que describen una secuencia de pasos detallados que deben seguirse para comprobar que el software funciona según lo esperado. Cada caso de prueba establece las condiciones iniciales específicas y define los datos de entrada necesarios para ejecutar una acción o escenario determinado, con el fin de obtener un resultado concreto y predecible.

Su propósito principal es detectar la mayor cantidad posible de defectos, optimizando el tiempo y el esfuerzo dedicado tanto a su diseño como a su ejecución. Mientras los requerimientos del sistema no cambien, los casos de prueba pueden reutilizarse indefinidamente, garantizando consistencia en las evaluaciones.
Una característica esencial de los casos de prueba es que deben ser REPRODUCIBLES: si un defecto no puede reproducirse siguiendo un caso de prueba documentado, entonces no puede considerarse un defecto validado. Esto asegura trazabilidad, control y confiabilidad en el proceso de verificación del software. Se derivan de diferentes fuentes:

- Documentos del cliente
- Información relevada
- Especificaciones de programación
- Código

Caso de prueba es el artefacto más importante de la etapa de prueba de aca sale el testing sistemático y metodológico, es un testing que encuentra defectos, para que encuentre defectos se tiene que reproducir. Un defecto que no se puede reproducir no es un defecto. Tenes que saber que volver a hacer para que salte ese defecto. Los de experiencia, diseñan los casos de prueba los mas inexpertos, ejecutan estos casos de prueba. Es mas detallado y especifico que en una prueba de US. No se puede hacer pruebas exhaustivas (osea todas las combinaciones)

Reporte de incidentes o defectos: Luego de la ejecución de los casos de prueba, se elabora un reporte, indicando cuáles fueron los casos de prueba que han pasado, y aquellos en los que se ha encontrado defectos, indicando cuáles fueron esos defectos encontrados y cómo reproducirlos para su detección y corrección. Este artefacto permite a los desarrolladores corregir esos defectos, para enviarlos nuevamente a testing.

Informe final: Suele contener métricas e información final del incremento del producto, se reporta cuántos ciclos y la cantidad de errores por ciclo, métodos, tipos de prueba utilizados, etc. Este es el único artefacto que es negociable en algunas organizaciones informales, se acuerda en la contratación del trabajo.
Tiene utilidades estadísticas.

El testing y el ciclo de vida: La relación entre el testing y el ciclo de vida del desarrollo del software depende del enfoque utilizado. En un ciclo de vida secuencial, las pruebas se realizan al final del proceso, una vez que el producto completo ha sido entregado para su validación. Esto implica que los defectos suelen detectarse en etapas tardías, lo que puede aumentar los costos y tiempos de corrección (recordar que cuánto antes se detectan los errores, menor es el costo y esfuerzo para solucionarlo).
En cambio, en un ciclo de vida ágil (iterativo e incremental), el testing se lleva a cabo en cada iteración o incremento del producto. Esto permite detectar y corregir errores de manera temprana, mejorando la calidad continua del software. Sin embargo, este enfoque también presenta un desafío: la integración de los incrementos puede introducir nuevos defectos, lo que exige una verificación constante y pruebas de regresión para asegurar la estabilidad del sistema

Estrategias de prueba: Las estrategias de prueba se aplican con el objetivo de evaluar la mayor cantidad posible de funcionalidades del software utilizando la menor cantidad de

casos de prueba, optimizando así el uso del tiempo y los recursos disponibles. Dado que el diseño y la ejecución de pruebas implican costos y esfuerzo, es fundamental seleccionar enfoques que maximicen la cobertura y la eficiencia.
Existen dos enfoques principales: caja negra y caja blanca. Cada uno tiene sus propias fortalezas y limitaciones: algunos métodos son más adecuados para ciertos tipos de pruebas que para otros. Por este motivo, la mejor estrategia no consiste en utilizar un único enfoque, sino en combinar diversas técnicas que se complementan entre sí, permitiendo realizar un testing más completo, eficiente y confiable

Caja negra: Es una estrategia de testing en la que no se tiene acceso a la estructura interna o al código del software, sino que se analiza su comportamiento únicamente a partir de las entradas y salidas. Es decir, se trata al sistema como una caja negra que recibe datos y devuelve resultados, sin conocer cómo se procesan internamente.
El proceso consiste en ingresar distintos datos al sistema y comparar los resultados obtenidos con los resultados esperados. Se prueban tanto las entradas válidas como las no válidas, evaluando además diferentes secuencias de transacciones que puedan influir en el comportamiento del sistema. Esta estrategia se divide en dos enfoques:
Métodos basados en especificaciones: Utilizan la documentación del producto (como requerimientos o casos de uso) para definir los casos de prueba

- Partición de equivalencias: Este método consiste en dividir los datos de entrada en clases de equivalencia, es decir, subconjunto de valores que debería producir el mismo resultado. En lugar de probar todos los valores posibles, se selecciona un valor representativo por cada clase. Por ejemplo, si un formulario acepta edades entre 18 y 65 años, se pueden definir tres clases; edad menor a 18 (inválida), edad entre 19 y 65 (válida), edad mayor a 65 (inválida)
- Análisis de valores límites: Variante del método anterior que se enfoca en probar los límites de las clases de equivalencia, ya que los errores suelen presentarse en esos extremos, en el ejemplo anterior se probarían edades como 17,65, etc

Métodos basados en experiencia: Dependen del conocimiento, la intuición y la experiencia del testes para definir las pruebas más propensas a detectar errores

- **Adivinanzas de defectos:** Se elabora una lista posible de defectos o situaciones problemáticas basándose en la experiencia previa y se diseñan pruebas para intentar reproducirlos
- **Testing exploratorio:** El tester aprende, diseña y ejecuta pruebas simultáneamente. A medida que va utilizando el sistema, genera nuevas ideas de prueba basadas en su comprensión, creatividad e intuición

Partición de equivalencias - procedimiento paso a paso:
1. Identificar condiciones externas de entrada y salida: Son los datos o factores que intervienen en la funcionalidad a probar.

- **Ejemplos de entradas:** campos de texto, fechas, coordenadas, archivos o señales de sensores
- **Ejemplos de salidas:** mensajes de pantalla, listados, advertencias o señales emitidas

2. Definir subconjuntos de valores posibles (válidos y no válidos): Para cada condición externa, de forma que los valores dentro de un mismo subconjunto produzcan resultados equivalentes

- Los subconjuntos inválidos suelen asociarse con mensajes de error o advertencia
- La unión de todos los subconjuntos debe cubrir todo el universo posible de valores

3. Construir los casos de prueba: Seleccionando un valor representativo de cada subconjunto identificado. Aspectos a considerar en los casos de prueba

- Prioridad
    - **Alta:** para los casos críticos o caminos felices del negocio
    - **Baja:** para validaciones secundarias, como errores de formato o campos vacíos
- **Nombre del caso de prueba:** Debe describir claramente el escenario. Ejemplo: “Ingresar al sitio web con edad mayor a 19 años”
- **Precondiciones:** Son los requisitos previos que deben cumplirse para ejecutar la prueba, como un usuario autenticado o una fecha específica. Ejemplo “El usuario X está logueado con el rol de administrado”

Pasos:

- Comienzan indicando la acción que activa la funcionalidad “El usuario selecciona la opción X”
- Continúan con las instrucciones realizadas “El usuario ingresa…” o “El usuario selecciona…”
- Deben estar redactados de forma precisa y sin ambigüedades, ya que otra persona debe poder reproducirlos
- Generalmente finalizan con una acción de confirmación

Resultado esperado: Describe lo que el sistema debería mostrar o realizar como consecuencia de los pasos ejecutados. Debe ser específico y verificable, por ejemplo: El sistema muestra el mensaje de error ‘Debe ingresar una fecha válida’
Cada resultado esperado debe estar claramente asociado con el paso que lo provoca, garantizando trazabilidad y claridad durante la ejecución de la prueba

Caja blanca: En este tipo de pruebas, se tiene acceso al código fuente, al pseudocódigo o incluso a los diagramas de flujo del sistema. Este conocimiento permite diseñar casos de prueba que maximicen la detección de defectos utilizando la mejor cantidad posible de pruebas. Sin embargo, presenta dos limitaciones importantes: la primera es que el número de caminos lógicos dentro del código puede ser extremadamente alto, lo que hace inviable probarlos todos en un tiempo razonable. La segunda es que, aun cuando se realicen pruebas exhaustivas de todos los caminos posibles, esto no garantiza que el programa cumpla correctamente con todas las especificaciones. Además, este enfoque no permite identificar caminos faltantes ni detectar errores relacionados con datos sensibles.
Existen distintos niveles de cobertura dentro de la prueba de caja blanca, los cuales se refieren a las diferentes formas de recorrer los caminos lógicos del código para verificar la correcta implementación de una funcionalidad

Tipos de prueba

Smoke test: Es un tipo de prueba inicial que se realiza para comprobar que el producto de software no presente fallas graves o críticas que impidan su funcionamiento básico. Su

objetivo es asegurar que el sistema esté lo suficientemente estable como para iniciar el proceso formal de pruebas.
Se lleva a cabo antes del comienzo del ciclo 0 y consiste en una ejecución rápida y superficial del sistema, destinada a verificar que las funciones principales operen correctamente. En caso de detectarse errores importantes en esta etapa, el testing formal se detiene hasta que dichos problemas sean corregidos, evitando así invertir tiempo y recursos en un producto que aún no está listo para ser evaluado en profundidad

Testing funcional: Tiene como objetivo verificar que el software se comporte exactamente como se especifica en la documentación, asegurando que cumpla con todas las funcionalidades y características definidas. Este tipo de prueba se basa en los requerimientos funcionales y en el flujo de los procesos de negocio, validando que el sistema responda correctamente ante distintas entradas y situaciones esperadas. Existen dos enfoques principales dentro del testing funcional:

- **Basado en requerimientos:** Se centra en probar funcionalidades específicas del sistema. Cada prueba verifica el cumplimiento de un requerimiento definido en la ERS o en los criterios de aceptación de una US. Su propósito es comprobar que cada requisito individual funcione correctamente
- **Basado en procesos de negocio:** Evalúa el funcionamiento integral de un proceso completo dentro del sistema. En este caso, se valida que las distintas funcionalidades interactúen adecuadamente entre sí. Por ejemplo, en un proceso de venta, se probaría la búsqueda de un articulo, su selección, la facturación y el cierre de la operación

Testing no funcional: Se enfoca en evaluar cómo funciona el sistema, más que qué hace. Es decir, analiza los aspectos relacionados con el rendimiento, la estabilidad, la seguridad, la usabilidad y otros factores de calidad definidos en los requerimientos no funcionales.
Este tipo de pruebas suele ser más complejo, ya que depende en gran medida del entorno en el que se ejecuta el sistema, el cual debe ser lo más similar posible al entorno real del cliente. Sin embargo, existen características difíciles de reproducir completamente (como ancho de banda, la seguridad en condiciones reales o la performance en escenarios específicos), que pueden ser verificadas más adelante durante las pruebas de aceptación.
El testing no funcional incluye varios tipos de pruebas, entre ellas:

- **Performance:** evalúa los tiempos de respuesta y el comportamiento del sistema bajo escenarios esperados de carga y concurrencia. Es fundamental que el software supere este tipo de prueba
- **Carga:** además de medir el rendimiento, analiza cómo responden los recursos del sistema (procesador, disco, memoria, red) ante diferentes volúmenes de trabajo.
- **Estrés:** somete al sistema a condiciones extremas o fuera de lo habitual para observar su punto de falla y su capacidad de recuperación. Permite medir la robustez y estabilidad del software
- **Mantenibilidad:** determina si el producto puede evolucionar fácilmente. Se verifica la existencia de documentación técnica, manuales de configuración y la facilidad para detectar y corregir defectos
- **Usabilidad:** analiza la experiencia del usuario, comprobando que el sistema sea fácil, intuitivo y agradable de utilizar
- **Portabilidad:** valida que el software funcione correctamente en los distintos entornos o plataformas acordadas con el cliente

- **Fiabilidad:** mide la capacidad del sistema para ofrecer resultados consistentes y seguros a lo largo del tiempo
- **Interfaz de usuario:** verifica el correcto funcionamiento y diseño de las interfaces gráficas (GUI), las cuales suelen ser más complejas que las interfaces basadas en comandos
- **Configuración:** comprueba que el sistema pueda instalarse, configurarse y ejecutarse adecuadamente en diferentes contextos o ambientes técnicos

Test driven development - TDD: Es una metodología de desarrollo de software que propone escribir primero las pruebas unitarias antes de implementar el código del componente. La idea central es que, al definir las pruebas desde el inicio, se obtiene una comprensión más clara de los requerimientos y se reducen los errores durante la implementación. Su principio filosófico se resumen en: “sí no sé exactamente qué debo hacer, no puedo crear pruebas que lo validen”. TDD combina dos prácticas fundamentales:

- **Test first development:** En esta técnica, se crean primero las pruebas unitarias correspondientes a la funcionalidad que se desea implementar. Una vez definidas, el desarrollador escribe el código necesario para que dichas pruebas pasen correctamente. Este enfoque fomenta la modularización del código, ya que obliga a diseñar métodos y clases con una única responsabilidad, facilitando su mantenimiento y compresión. Además, puede incluir desde el inicio pruebas de integración, que verifican la interacción entre diferentes componentes del sistema
- **Refactoring:** Consiste en reestructurar el código existente sin alterar su comportamiento funcional. Su propósito es mejorar la legibilidad, reducir la complejidad y optimizar la mantenibilidad del software.

El refactoring es una práctica continua en TDD, ya que cada vez que se logra que una prueba pase, se revisa y mejora el código para hacerlo más limpio y eficiente
En conjunto, el TDD promueve un ciclo iterativo de tres pasos conocidos como red-green-refactor:

- **Red:** Escribir una prueba que falle
- **Green:** Escribir el código mínimo necesario para que la prueba pase
- **Refactor:** Mejorar el código sin romper las pruebas

Este proceso asegura un desarrollo más ordenado, confiable y con menor cantidad de defecto